

Microcontroller Project

by
Group 4

Zohaib Yasar

Suber Abdi

Zhixin Huang

Arhaan Ali

Makary Miszczuk

Abstract	3
Introduction	3

Chapter 1: Theory & Background	4
1.1 An Overview of Data Acquisition Systems.	4
1.2 The purpose of signal pre-conditioning in data acquisition systems	5
1.3 Principles of Frequency, Bandwidth and Signal Filtering	5
1.4 Principles, Operation, Practical Considerations, Limitations and Application of Common Devices, Circuits and Theorems.....	7
1.4.1 The Resistor	7
1.4.2 Ohm's Law	8
1.4.3 Capacitor	8
1.4.4 Potential Divider Circuit	9
1.4.5 Op-Amp (Operational Amplifier).....	9
1.4.6 Voltage Follower Circuit	10
1.4.7 Switches	10
1.4.8 Analogue to Digital Converter	11
1.4.9 Thermistor.....	12
1.5 Overview of SIMSCAPE and MATLAB	13
1.6 Overview of MPLABX	14
1.7 Overview of Fusion 360, the PCB design and schematic entry tool.....	14
1.8 Limitation, Benefits and Connectivity of PICSIM	15
1.9 The Necessity of Accurate Timing.....	15
1.10 An LCD's panel's timing and functionality	16
Chapter 2: Hardware	18
2.1 Overall System (block) Diagram of Solution	18
2.2 Micro-controller Circuit.....	19
2.3 Power Regulation Circuit	19
2.4 Analogue Front End (AFE) Circuit.....	20
2.5 LCD Driver Circuit.....	20
2.6 Input Select Mechanism	22
2.7 PCB Layout.....	22
Chapter 3: Software	24
3.1 Overall System Logic & Program Structure	24
3.2 Firmware Configuration for PIC16F877A.....	25
3.3 LCD Driver & Timing	28

3.4 Binary Coded Decimal (BCD) Conversion and Scaling	32
3.5 Input Selection	33
Chapter 4: Result & Testing.....	33
4.1 Behavioural Simulations of AFE.....	33
4.2 Testing Methodology and Test Vectors Used	34
4.3 Use of MPLABX simulations for testing and results.....	35
4.4 Virtualisation of solution on PICSIM and results	39
Conclusion	40
References.....	40
Appendices	42
A	42
B	43

Abstract

This report explores the fundamental principles of electronic components as well as going into detail about the hardware and software surrounding the data acquisition system we have made. We have managed to create a system that consists of a PIC16F877A that runs the software which will acquire, process and display the digital value of a signal, it is connected to an Analog Front-End circuit that has two sensors that will collect this information. There is also additional hardware such as a voltage regulator, reset circuit, oscillator circuit and an LCD Display It also presents and explains the development, results and testing done through software such as MATLAB, MPLABX, PICSIM and Fusion 360.

Introduction

This project was created to develop testing equipment that it can be use in thermal tests, this is so that it can be possible to record and spectate heat dissipation at two different areas. To successfully develop this project it required a PIC, LCD display, AFE circuit, and several other basic electronic components such as a power supply or an oscillator. Furthermore, this testing equipment has software that has been developed be run by the hardware.

Two NTC thermistors that are connected to separate AFE circuits will collect the analogue information where the analogue signal will go through signal conditioning. Afterwards this signal will go to the analogue to digital converter (ADC) on the PIC which will convert the analogue signal to a digital value, this digital value will then be

displayed on an LCD which has be programmed to switch between channels so it can display information/values from both NTC thermistors.

This report is divided into four chapters which is split between the theory and background of the project, the hardware, the software, and testing. In addition to this, it concludes with a summarisation of the results and achievements of the project, as well as two appendices which contains all the schematics and the codes created on MATLAB, Fusion 360 and MPLABX.

Chapter 1: Theory & Background

1.1 An Overview of Data Acquisition Systems.

Data acquisition system is when hardware and software combined is used to record and measure a physical event and convert it into digital data so it can be stored and analysed by a computer (Park and Mackay, 2003). A data acquisition system usually consists of software, hardware, signal conditioning, a computer, wiring, transducers and sensors (Park and Mackay, 2003).

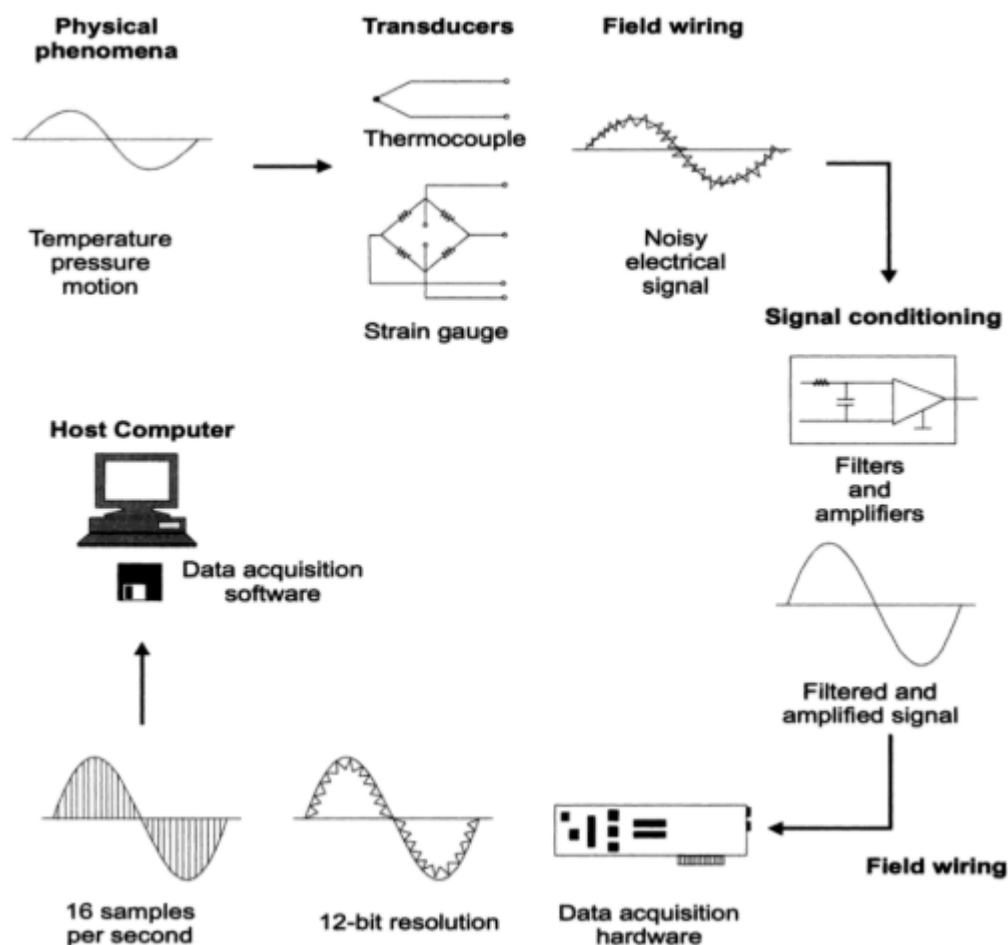


Figure 1: Image of a data acquisition system (Park and Mackay, 2003)

1.2 The purpose of signal pre-conditioning in data acquisition systems

Signal conditioning is when an analogy signal is manipulated in order to be processed further when converting an analogue signal to a digital signal. The process of signal conditioning can include processes such as filtering, attenuating and amplifying (Morris, 2001). The manipulation of a signal aims to improve the quality of the signal as well as attenuate or amplify some frequencies. Signal filters will remove a certain range of unwanted frequencies (Morris, 2001). The purpose of using signal conditioning is so that it can be converted into a digital signal during one of the steps in a data acquisition system (see Figure 1) so it can later go through the hardware and software parts (Park and Mackay, 2003).

1.3 Principles of Frequency, Bandwidth and Signal Filtering

Frequency is how many cycles or oscillations occur per second, this is measured in hertz (Hz) and the equation for this is $f = 1/T$ (Kyle Forinash and Wolfgang Christian).

When a signal is transmitted, it has a range of frequencies, the difference between the highest and lowest frequencies is called the bandwidth (Plonus, 2020). An example of this would be a signal with frequencies ranging from 500 Hz to 1 kHz is 499 Hz (Plonus, 2020).

Signal filtering is a process of selecting certain frequencies whilst completely or partially reducing the unneeded frequencies. This is to lessen the background noise and suppress signals that could interfere with the signals that need to be selected (Morris, 2001). There are different types of filters such as band-pass, band-stop, low-pass and high pass filters. The characteristics that these filters possess include order (steepness), type and cutoff frequency (Morris, 2001).

A band-pass filter only allows frequencies in a certain bandwidth, this can be seen in Figure 2 (Morris, 2001). A band-stop filter only weakens frequencies in a particular bandwidth; this can be seen in Figure 3 (Morris, 2001). A low-pass filter will only permit a low frequency to pass whilst weakening higher frequencies, which is seen in Figure 4 (Morris, 2001). A high-pass filter is the opposite of a low-pass filter where high frequencies are permitted whereas low frequencies are weakened, which is also seen in Figure 5 (Morris, 2001).

An RC filter is a filter that includes a resistor and capacitor in a parallel circuit and an active RC filter has an op-amp (an active component) in the circuit which will provide a filter for low frequencies, this is an active component because it draws power from an external source (RC Filter).

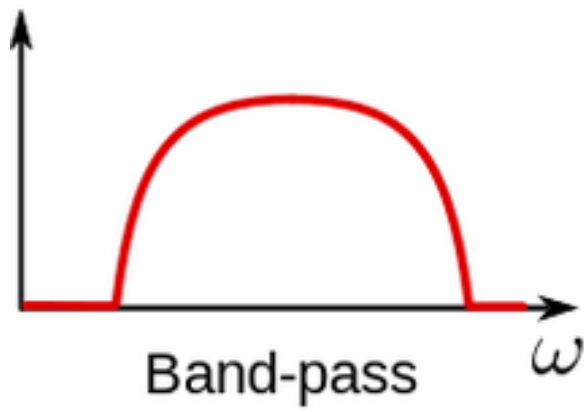


Figure 2: A frequency band that has gone through band-pass filtering (Dewesoft,2023)

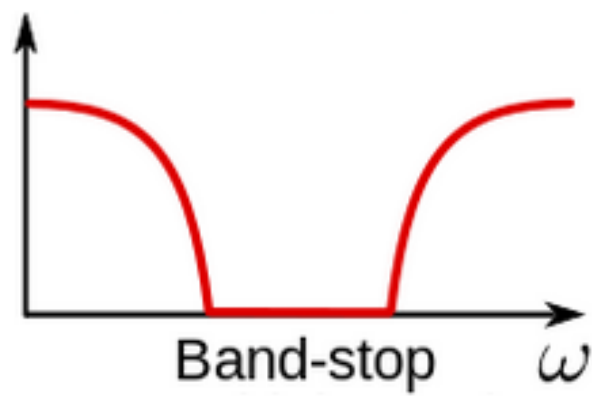


Figure 3: A frequency band that has gone through band-stop filtering (Dewesoft,2023)

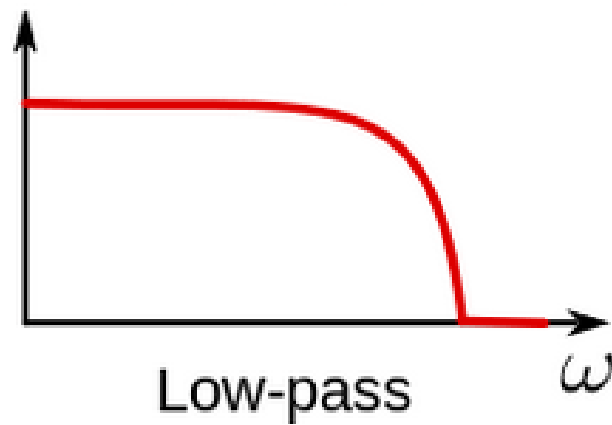


Figure 4: A frequency band that has gone through low-pass filtering (Dewesoft,2023)

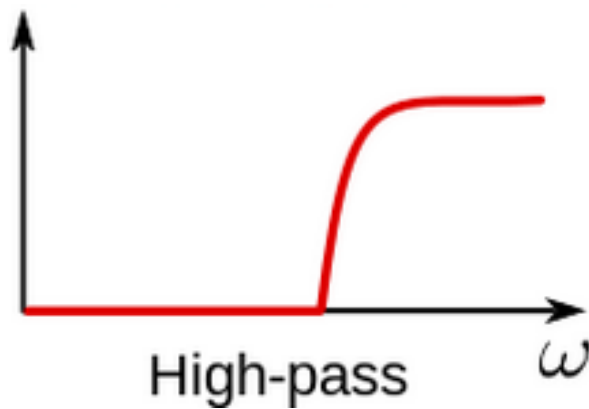


Figure 5: A frequency band that has gone through high-pass filtering (Dewesoft, 2023)

1.4 Principles, Operation, Practical Considerations, Limitations and Application of Common Devices, Circuits and Theorems

1.4.1 The Resistor

Resistors are a component in electronics which restrains the flow of a current in a circuit. There are many different types of resistors; such as fixed resistors (See Figure 6) and variable resistors (See Figure 7) (Horowitz and Hill, 2015). A fixed resistor keeps the same resistance value whilst a variable resistor (a potentiometer) can have its value changed, this is normally done by turning a shaft (Horowitz and Hill, 2015).

Resistors are a resourceful component, holding multiple purposes within a circuit such as;

- Limiting the current to an LED to change the level of brightness
- Measuring the current using Ohm's law
- Generating immediate voltages through potential dividers (Horowitz and Hill, 2015).

However, there are limits of using resistors in practical use. This is largely due to its thermal resistance (Horowitz and Hill, 2015). This is because if a resistor cannot dissipate power into its environment properly or if it is running near its own maximum value, it would significantly impact the reliability of the system in the long run. (RS Components, 2024).



Figure 6: An image of a fixed resistor (RS Components, 2024)



Figure 7: An image of a variable resistor (potentiometer) (RS Components, 2024)

1.4.2 Ohm's Law

Ohm's law is an equation that connects three changeable variables, comprising of the three; voltage, resistance and current. The relationships between the variables are shown in this equation $V = IR$ (Horowitz and Hill, 2015). Ohm's law in practical use is mostly for measurements through using a resistor to change the current which in turn changes the level of voltage, this can be seen by rearranging the equation ($R = V/I$, $I = V/R$) as these values share a proportional relationship (Horowitz and Hill, 2015). For example, if the resistance increases and current stays the same, the voltage will increase (Horowitz and Hill, 2015).

1.4.3 Capacitor

A capacitor is a device responsible for storing electrical energy of within the circuit, it contains two conductors that are close together and have opposing electric charges (Horowitz and Hill, 2015). Figure 8 shows a real-life capacitor. Capacitors are essential in practical uses like digital circuits, as they are necessary in preventing data loss in computer storages if power is lost in unforeseen circumstances (Horowitz and Hill, 2015). When it comes to filters, they are used to block electrical signals from damaging sensitive electrical components, this principle of blocking signals can also be applied to signal filters such as high-pass filter (Horowitz and Hill, 2015).



Figure 8: Image of a 100 µF Capacitor (RS Components)

1.4.4 Potential Divider Circuit

Potential divider circuit is when the input voltage is divided using two resistors, the output voltage becomes a fraction of input voltage, this can be calculated and shown from this equation for this circuit: $V_{out} = V_{in} (R_2 / R_1 + R_2)$ (Horowitz and Hill, 2015). The purpose of this circuit is to reduce the voltage in the circuit, which can be used for power regulations. An ideal practical use of a potential divider circuit would be using at low frequencies as signal attenuators (Horowitz and Hill, 2015).

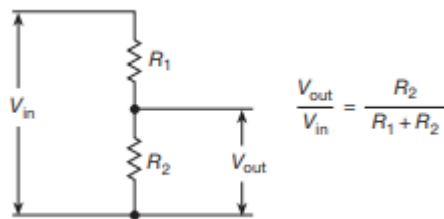


Figure 9: Potential Divider Circuit (Horowitz and Hill, 2015)

1.4.5 Op-Amp (Operational Amplifier)

An operational amplifier is an electronic component with two inputs that it amplifies the difference between each other and outputs singular result (Jung, 2004). In practical use an op-amp can be utilised in signal conditioning, an analogy filter or a signal amplifier (Jung, 2004).

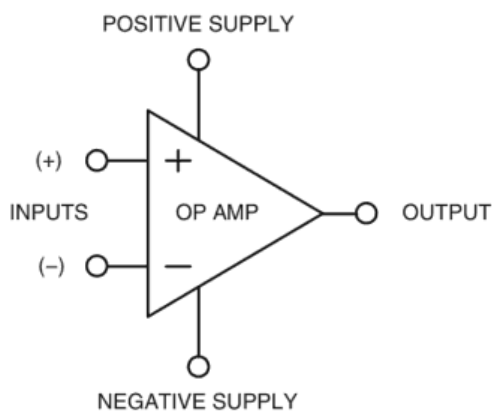


Figure 10: A diagram of an Op-Amp (Jung, 2004)

1.4.6 Voltage Follower Circuit

A voltage follower (otherwise known as either a voltage buffer or unity gain amplifier) is a circuit which input voltage is being followed by the output voltage, this is done by using wires to connect the non-inverting input (+) to the input signal and connecting the inverting input (-) to the output (Jung,2004).

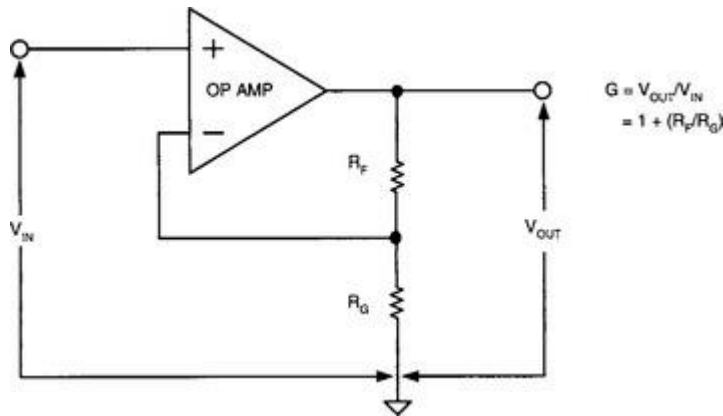


Figure 11: Image of a Voltage Follower Circuit (Jung,2004)

1.4.7 Switches

There are passive electronic components such as electronic switches (also known as solid state switches) which are designed to interrupt the flow of current in a circuit, a switch has two stages on and off (Ravi Teja, 2024). Electronic switches have no physical moving parts, so no physical contact is necessary, there are different types such as MOSFETs, IGBTs, SCRs and TRIACs (Ravi Teja, 2024).

The most common switch and the one we plan to use is the MOSFET (Metal Oxide Semiconductor Field Effect Transistor), it is a device which has three terminals called gate (input), drain (output) and source (common) (Ravi Teja, 2024). The on and off state of this switch is decided by controlling resistance between the source and drain, the input voltage from the gate to the source (Ravi Teja, 2024). The electronic switch we use will control what channel is being read by the microcontroller.

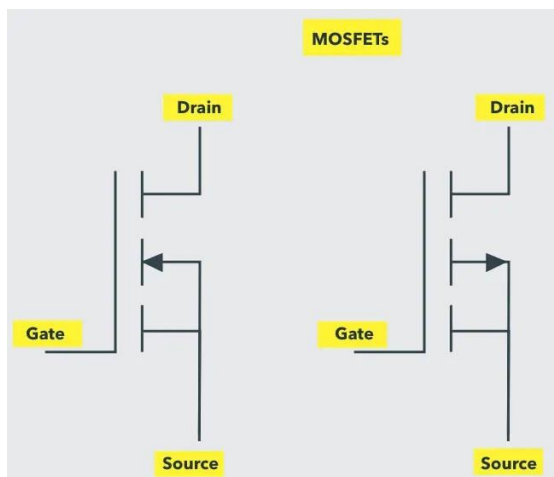


Figure 12: Image of the layout of MOSFET (Ravi Teja, 2024)



Figure 13: Image of a MOSFET (MOSFET,2024)

1.4.8 Analogue to Digital Converter

Analogue to digital converters is a way to convert analogue information into digital information (digital signals) so that it can be used in computer software, digital signals are a sequence of discrete values whilst analogue signals are sequence of continuous values (see Figure 14) (Arrow, 2023). There is step in converting analogue to digital known as sampling and quantifying, the signal will first be sampled, then the resolution of the signal will be determined by quantifying it and lastly binary values will be set so the software will be able to read the signal (Arrow, 2023).

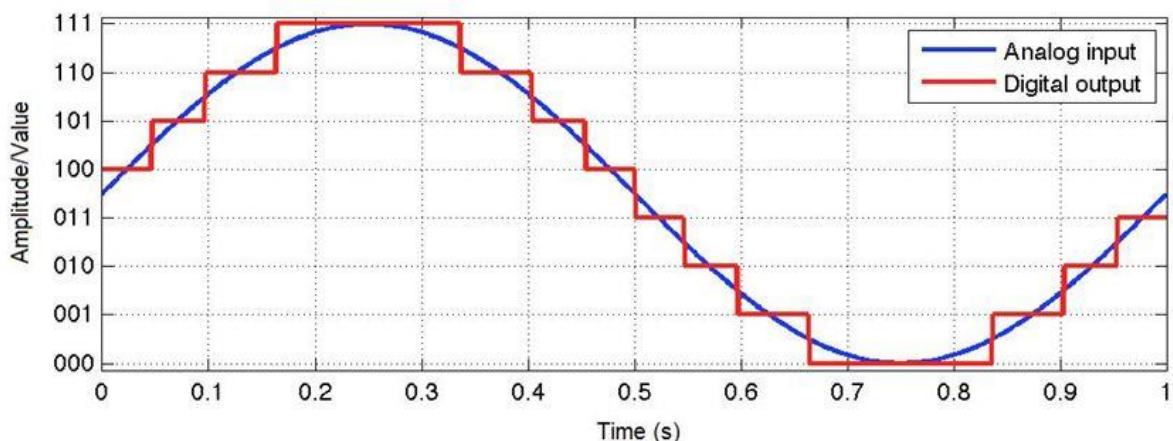


Figure 14: An image of an analogue signal turning into a digital signal (Arrow, 2023)

The ADC in our microcontroller (PIC16F877A) is built in, it is used through the assembly code in MPLABX where the ADC is initialised then the input is collected and stored. In Figure 15, the assembly code sets up the input for the ADC, whilst in Figure 16, the input is being read and stored so it can be sent to the LCD display (Print).

```

; PROGRAM BEGINS ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
org    0x00          ; Default start address
NOP ; required for ICD model
; Port & display setup.....
BANKSEL TRISC        ; Select bank 1
CLRF   TRISD         ; Display port is output
movlw  0x0f
movwf  TRISA
CLRF   TRISE         ; Display port is output
MOVLW  B'00000010'   ; Analogue input setup code
MOVWF  ADCON1        ; Left justify result,
; Port A = analogue inputs
BANKSEL PORTC        ; Select bank 0
CLRF   PORTD         ; Clear display outputs
CLRF   PORTE         ; Clear display outputs
MOVLW  B'01000001'   ; Analogue input setup code
MOVWF  ADCON0        ; f/8, RA0, done, enable
CALL   inid          ; Initialise the display
BCF    PORTB,RB0
CALL   putLCD
GOTO   start

```

Figure 15: Screenshot of Assembly Code

```

; SUBROUTINES ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
; Read ADC input and store .....
getADC BSF    ADCON0,GO    ; start ADC..
wait   BTFSC  ADCON0,GO    ; ..and wait for finish
        GOTO   wait
        MOVF   ADRESH,W    ; store result high byte
        RETURN

; Convert input to decimal .....
condec MOVWF  ADbin        ; get ADC result
        CLRF   huns        ; zero hundreds digit
        CLRF   tens        ; zero tens digit
        CLRF   ones        ; zero ones digit
; Calculate hundreds.....
        BSF    STATUS,C    ; set carry for subtract
        MOVLW  D'100'      ; load 100
sub1    SUBWF  ADbin        ; and subtract from result
        INCF   huns        ; count number of loops
        BTFSC  STATUS,C    ; and check if done
        GOTO   sub1        ; no, carry on
        ADDWF  ADbin        ; yes, add 100 back on
        DECF   huns        ; and correct loop count
; Calculate tens digit.....
        BSF    STATUS,C    ; repeat process for tens
        MOVLW  D'10'       ; load 10
sub2    SUBWF  ADbin        ; and subtract from result
        INCF   tens        ; count number of loops
        BTFSC  STATUS,C    ; and check if done
        GOTO   sub2        ; no, carry on
        ADDWF  ADbin        ; yes, add 100 back on
        DECF   tens        ; and correct loop count
        MOVF   ADbin,W     ; load remainder
        MOVWF  ones        ; and store as ones digit
        RETURN            ; done

```

Figure 16: Screenshot of Assembly Code

1.4.9 Thermistor

Thermistors are resistors which resistance is dependent on the temperature of the environment around it, this is great for a data acquisition system (dwyeromega). There are two types of thermistors a NTC (Negative Temperature Coefficients) and PTC (Positive Temperature Coefficients) thermistor, as temperature increases the

resistance of a PTC increases, the NTC is the opposite of it (dwyeromega). The practical application of thermistors is where electronic noise is attendance or in environments of extreme condition (dwyeromega).

For our hardware, we will be using two NTC thermistors, if our resistance goes down when temperature increase our voltage does down, this means that higher temperatures for our thermistors will mean lower voltages displayed on our LCD display.



Figure 17: Image of a Thermistor (Rs-Online)

1.5 Overview of SIMSCAPE and MATLAB

Matlab is programming platform commonly used to design, develop, and operate a series of wide ranges of engineered systems in real life applications (The MathWorks. Inc, 2025). With over 5 million users, with students from Primary schools and universities to expert engineers and scientists at major companies, startups, and accelerator programs, MATLAB poses itself a user-friendly platform readily available for a wide range of diverse users (The MathWorks. Inc, 2025). Many billion-dollar industries employ MATLAB in developing their products in:

- Autonomous and connected vehicles
- Electric cars and aircraft
- Marine vehicles
- Spacecraft
- Mobile devices and wireless networks
- Manufacturing machinery
- Renewable power generation machines, energy management, and storage systems
- Smart medical devices - (Mathswork, 2025)

MATLAB features built in features and designated algorithms which allows users to immediately start project work and automatically generate embedded code (The MathWorks. Inc, 2025). MATLAB code can be integrated with other programming languages such as C/C++, Java, .NET, Python: it can even seamlessly adapt to interfaces like Microsoft Excel (The MathWorks. Inc, 2025).

Simscape is an add-on software to Simulink that allows its user to create virtual models of a physical system like an electronic circuit (The MathWorks. Inc, 2025). A practical usage of this software is to design a schematic for a project before making the actual circuitry (The MathWorks. Inc, 2025). This allows for a convenient way to simulate any large scaled system or quickly work through and attempt multiple design ideas (The MathWorks. Inc, 2025).

1.6 Overview of MPLABX

MPLABX is a software program used to develop and debug designs for digital signal controllers and microcontrollers, it holds many connectivity points with any hardware that can run assembly code (Microchip, 2024). A user of MPLABX would develop and run their code on the IDE, they would also use the tools like the simulator (logic analyser) or target memory view (to see the values of the registers) (Microchip, 2024). There are also other features like export as a hex file so that the code can be run in simulation programs such as PICSIM (Microchip, 2024).

SFRs x

Stimulus

File Registers

Logic Analyzer

Address /

Name

Hex

Decimal

Binary

Char

	WREG	0x00	0	00000000	'.'
000	INDF	0x00	0	00000000	'.'
001	TMR0	0x00	0	00000000	'.'
002	PCL	0x00	0	00000000	'.'
003	STATUS	0x00	0	00000000	'.'
004	FSR	0x00	0	00000000	'.'
005	PORTA	0x00	0	00000000	'.'
006	PORTB	0x00	0	00000000	'.'
007	PORTC	0x00	0	00000000	'.'
008	PORTD	0x00	0	00000000	'.'
009	PORTE	0x00	0	00000000	'.'
00A	PCLATH	0x00	0	00000000	'.'
00B	INTCON	0x00	0	00000000	'.'
00C	PIR1	0x00	0	00000000	'.'
00D	PIR2	0x00	0	00000000	'.'
00E	TMR1	0x0000	0	00000000 00000000	'..'
00E	TMR1L	0x00	0	00000000	'.'
00F	TMR1H	0x00	0	00000000	'.'
010	T1CON	0x00	0	00000000	'.'
011	TMR2	0x00	0	00000000	'.'
012	T2CON	0x00	0	00000000	'.'

Figure 18: A screenshot of the analytical features of MPLABX (SFRs, Stimulus, FRs, Logic Analyser)

1.7 Overview of Fusion 360, the PCB design and schematic entry tool

Fusion 360 is a software platform that allows its user to design a schematic for a PCB as well as utilising the PCB layout tools to place components and route connections between them. In practical use this allows for less risk and error, the

ease of improving design, time and cost saving and better collaboration between individuals (Autodesk).

Autodesk Fusion 360 features schematic diagram labels which is used to label components of the circuits and specify its name, its value and even PIN numbers on a chip. uses standardised scientific symbols to represent components within the circuit (Autodesk). This enables it to be more accessible and widely industry recognised. This allows any new users easily build circuits and electrical designs within the system and nurture them into becoming experts (Autodesk). We use Fusion 360 to build our schematic circuits for our PCB design and customise components to our own specifications and requirements (Autodesk).

In our PCB we used Fusion 360 drew the following:

- AFE (Analogue Front End) with Thermistors
- Microcontroller Circuit
- LCD Display
- Reset Circuit
- Oscillator Circuit
- Power Supply
- Voltage Regulator
- The connections between the hardware

The AFE is used so the inputs from the sensors are conditioned before it is sent to the PIC microcontroller for further processing (Autodesk). The AFE will bias, scale and filter the sensor signal from the input. It contains a Voltage divider and an amplifier circuit to allow processing signals (Autodesk).

1.8 Limitation, Benefits and Connectivity of PICSIM

PICSIM (Programmable IC Simulator Laboratory) is an emulation software used to see how assembly code from MPLABX that has been exported as a hex file will work without the usage of any real hardware (Luis Gambôa, 2024). This software can support multiple devices such as LEDs and electronic switches for inputs and outputs. PICSIM is available on all major operating systems such as Linux, MacOS and Windows (Luis Gambôa, 2024).

PICSIM does have some limitations with the online version where it is advised to use 8Mhz clocks and below (Luis Gambôa, 2024). Additionally, when it comes to connectivity PICSIM can load hex files relevant to whatever board and microcontroller has been set (Luis Gambôa, 2024).

1.9 The Necessity of Accurate Timing

When it comes to embedded systems like microcontrollers, accurate timing is important so that devices can function properly (Medium, 2024). In this system we have clocks which are signals created by oscillators that give microcontrollers like

the PIC16F877A a reference for time (Medium, 2024). These clocks for time are tracked and allows all the functions and ongoing of the system to be synchronised (Medium, 2024). There also registers in the microcontroller that can be used to measure and record time intervals; these are known as timers (Medium, 2024).

In our software we have used this to cause a set of delays (15_ms, 1s and 5s), the 15ms delay is necessary for the initialisation of the LCD display. The remaining delays are for the rest of the program especially when switching channels, there is a delay between the messages.

1.10 An LCD's panel's timing and functionality

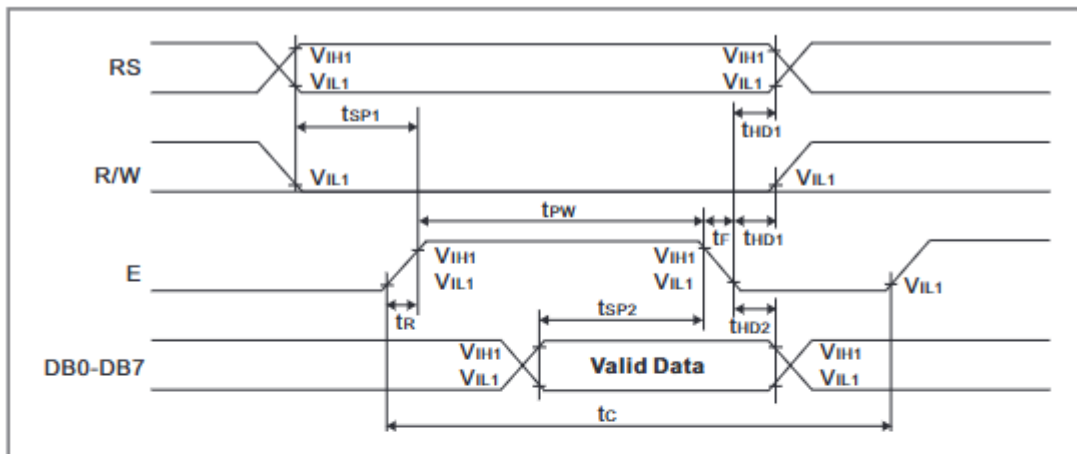
The LCD display that we are using is HD44780 16x2 and there are four group of pins relevant to timing (see Figure 19) (FORDATA, 2005):

- RS - Register Select Signal
- R/W - Data Read/Write
- E - Enable Signal
- DB0-DB7 - Data Line Bus

These pins are how the data travel between the LCD Display and whatever hardware (usually a microcontroller) it is connected to (FORDATA, 2005). The LCD display also has a lot of features such as an interface that can either be 4-bit or 8-bit as well as low power consumption (FORDATA, 2005). There are also various commands such as:

- Display Clear
- Return Home
- On/Off cursor
- Shift Display or Curser
- Read/Write Display

6.3.1 WRITE MODE TIMING DIAGRAM



6.3.2 READ MODE TIMING DIAGRAM

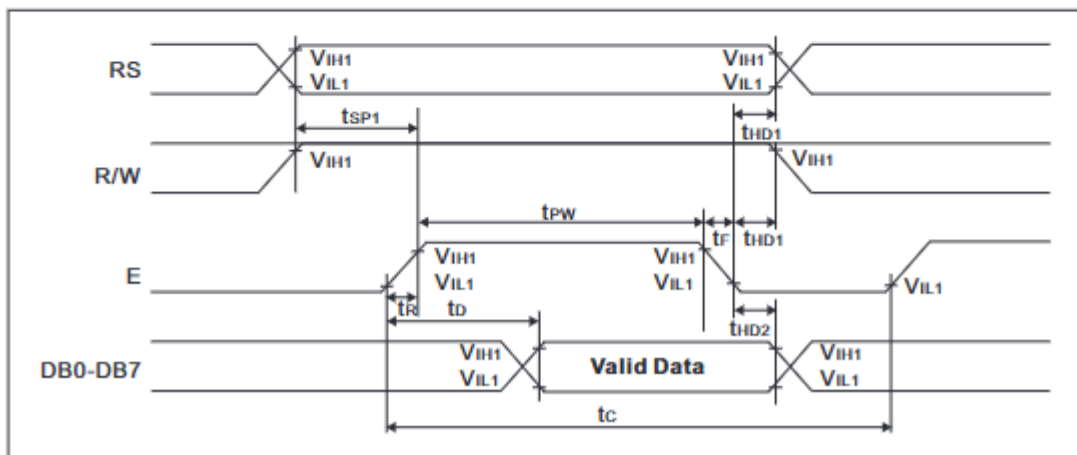


Figure 19: The Timing Diagram of the LCD Display (FORDATA, 2005)

Chapter 2: Hardware

2.1 Overall System (block) Diagram of Solution

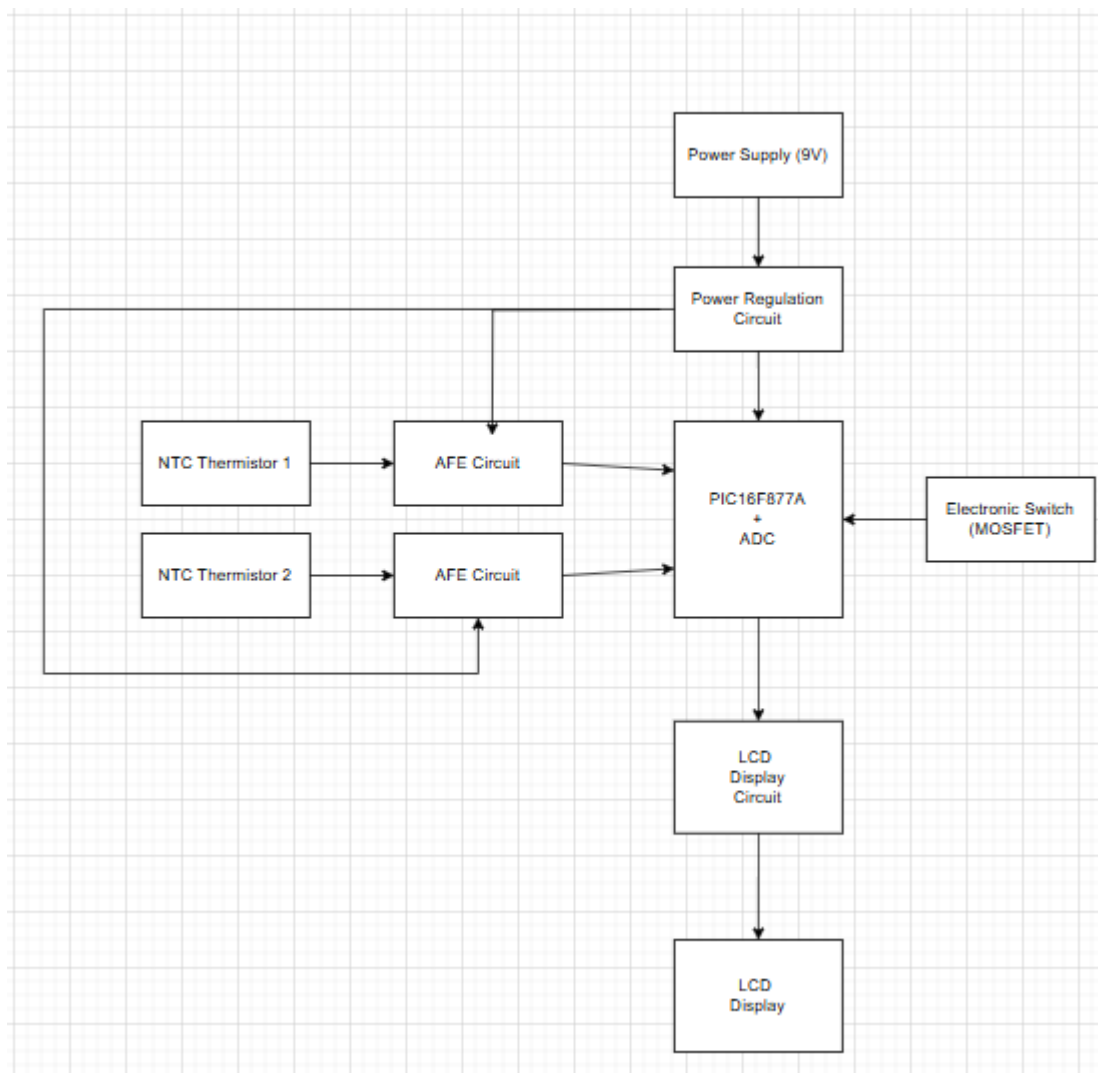


Figure 20: Block Diagram of Solution

This solution starts off with a 9V power supply which outputs the voltage for all the circuits. The voltage first goes through a power regulation circuit before going to any other electronic components. The rest of the hardware side of the solution contains two NTC thermistors (the sensors) which collect the data (signals) and is sent to the AFE (Analogue Front End) where signal conditioning takes place, which the signals become filtered. Afterwards, this data gets sent to the analogue to digital converter (ADC) in the PIC (microcontroller) where the data is converted to a digital value.

These digital values go through the LCD display circuit where it appears as readable values on the LCD display. The electronic switch controls which channel the microcontroller reads. In our case, we opted for MOSFET as our switch. After the digital values go through the software, they are sent to the LCD display and displayed as instructed by our constructed software.

2.2 Micro-controller Circuit

We opted for a 40 pin micro-controller (PIC16F877a), a diagram of this can be seen in Figure 21. When it comes to the firmware for this microcontroller, we were able to have eight inputs for the registers, the microcontroller is where the hardware is all connected, and the software is.

The microcontroller is connected to the power regulation circuit by VDD (pin 11). The microcontroller is connected to the LCD display by through the pins 21 to 30 to the pins on the LCD, D0 to D7. The microcontroller is connected to the AFE circuit that we designed through AN0 and AN1 (pin 2 and 3) for each thermistor/channel. The PCB layout that we designed in Section 2.7 shows in detail how everything is connected. We have two more circuits connected to the microcontroller circuit which is the reset circuit connected to the PIC through the MCLR pin (1), and the oscillator circuit which is connected through pin 13 (for input) and pin 14 (for output).

40-Pin PDIP

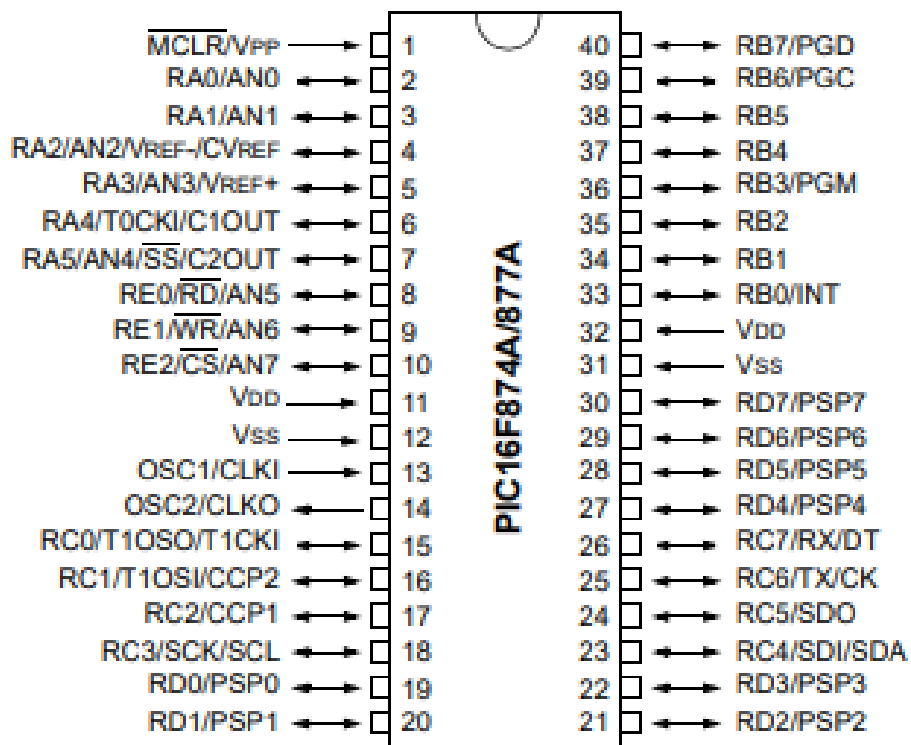


Figure 21: The pin layout of the microcontroller (PIC16F877A) (Microchip, 2003)

2.3 Power Regulation Circuit

A power regulation circuit is a safety measure which takes an input voltage (it may be unfixed or too high) from a power supply and creates a fixed output voltage (Monolithic Power). The type of voltage regulator circuit that we have chosen to use is a linear regulator which uses an op-amp (Monolithic Power). This will step-down the voltage and will always create an output voltage that is lower than the voltage from the power supply, this is great if the power supply voltage is way too high for the

rest of the electrical components (Monolithic Power). We have chosen to use a power regulation circuit to lower the voltage from 9 volts to 5 volts.

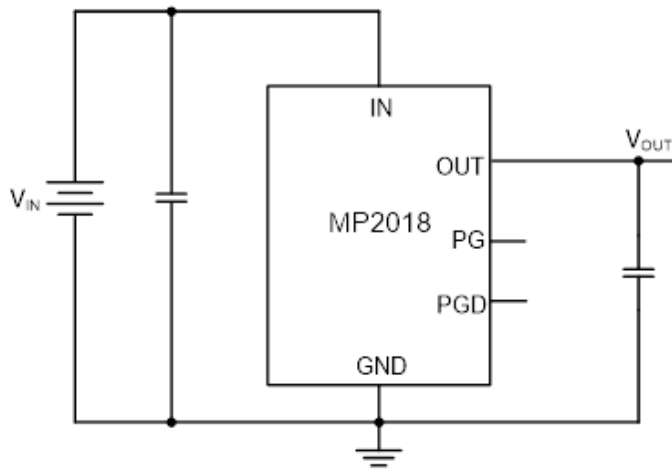


Figure 22: An image of a linear voltage regulator circuit (Monolithic Power)

2.4 Analogue Front End (AFE) Circuit

The analogue front end (AFE) is a group of circuits that perform signal conditioning and pre-processing (Avnet, 2023). It is located between the sensor (or other analogue signal source) and the subsequent digital processing unit (Avnet, 2023). Its primary function is to convert weak, noisy sensor signals into analogue signals with sufficient signal-to-noise ratio and linearity to meet the requirements of the back-end system or to directly drive an analogue-to-digital converter (Avnet, 2023). It is common practice to incorporate capacitors and/or inductors for the purpose of filtering (Avnet, 2023). Resistors are to be included, incorporating a potential divider network to act as a resistance to a voltage converter, or as a passive scaling stage (Avnet, 2023). The proposed divider network has the potential to act as a resistance to a voltage converter, or alternatively as a passive scaling stage (Avnet, 2023).

2.5 LCD Driver Circuit

The LCD Driver Circuit is connected to PIC16F877A (microcontroller) by wires attached to the assigned pins on the module and PIC, in Figure 23 details where the pins can be assigned (FORDATA, 2005). The pins on the microcontroller like RB1 (see Figure 21) that were used in the assembly code are all connected to pins, for example, RB0 to D0, RB1 to D1, etc. VDD is connected to the power regulation circuit, VSS is connected to GND.

8. PIN ASSIGNMENT

PIN	SYMBOL	FUNCTION
1	Vss	GND
2	Vdd	Power supply for LCM (+3.0V)
3	NC	No Connection
4	RS	Register Select Signal
5	R/W	Data Read / Write
6	E	Enable Signal
7-14	DB0 - DB7	Data bus line
15	NC	No Connection
16	NC	No connection

Figure 25: Explanation of the function of the pins of the LCD display (FORDATA, 2005)

2.6 Input Select Mechanism

For our inputs, we had chosen to use RB0 which act as our physical button that will make the LCD display switch between thermistor 1 and 2. We also require a physical switch for the MCLR (pin 1 on PIC) which is connected to the reset circuit and the button that we have chosen for RB0 and MCLR pin is a MOSFET electronic switch. We have also decided to make AN0 and AN1 (Pin 2 and 3) our input between the AFE circuit and the microcontroller.

2.7 PCB Layout

We used Fusion 360 to design our circuits and draw the pin connectivity points between each component and circuits. We built the following in our PCB layout:

- Oscillating Circuit
- LCD Display
- Reset Circuit
- Sensor Input Device
- Microcontroller and its attached displays and switches
- Power Circuitry and a Voltage Regulator

We designed the AFE (Analogue Front-End) using the Simscape tool in MATLAB and then recreated the circuits in our schematic on Fusion 360. We drew the voltage divider circuit, the amplifier circuit and the low pass filter. We used the same EU resistor variant throughout the circuits to maintain standardisation of our circuits and optimise efficiency whilst avoiding physical compatibility issues and varied power handling levels in our circuits (Quest Components, 2019). This would be due to the given fact that both different resistors are different variants and have their own maximum amount of electrical power they can safely dissipate before overheating or

failing (Quest Components, 2019). For example, a through-hole resistor from the EU and a surface mount resistor from the US hold different properties that can disrupt the flow of our circuit and potentially cause delays, Even though a 100Ω resistor from the EU will behave the same as a 100Ω resistor from the US (Quest Components, 2019).

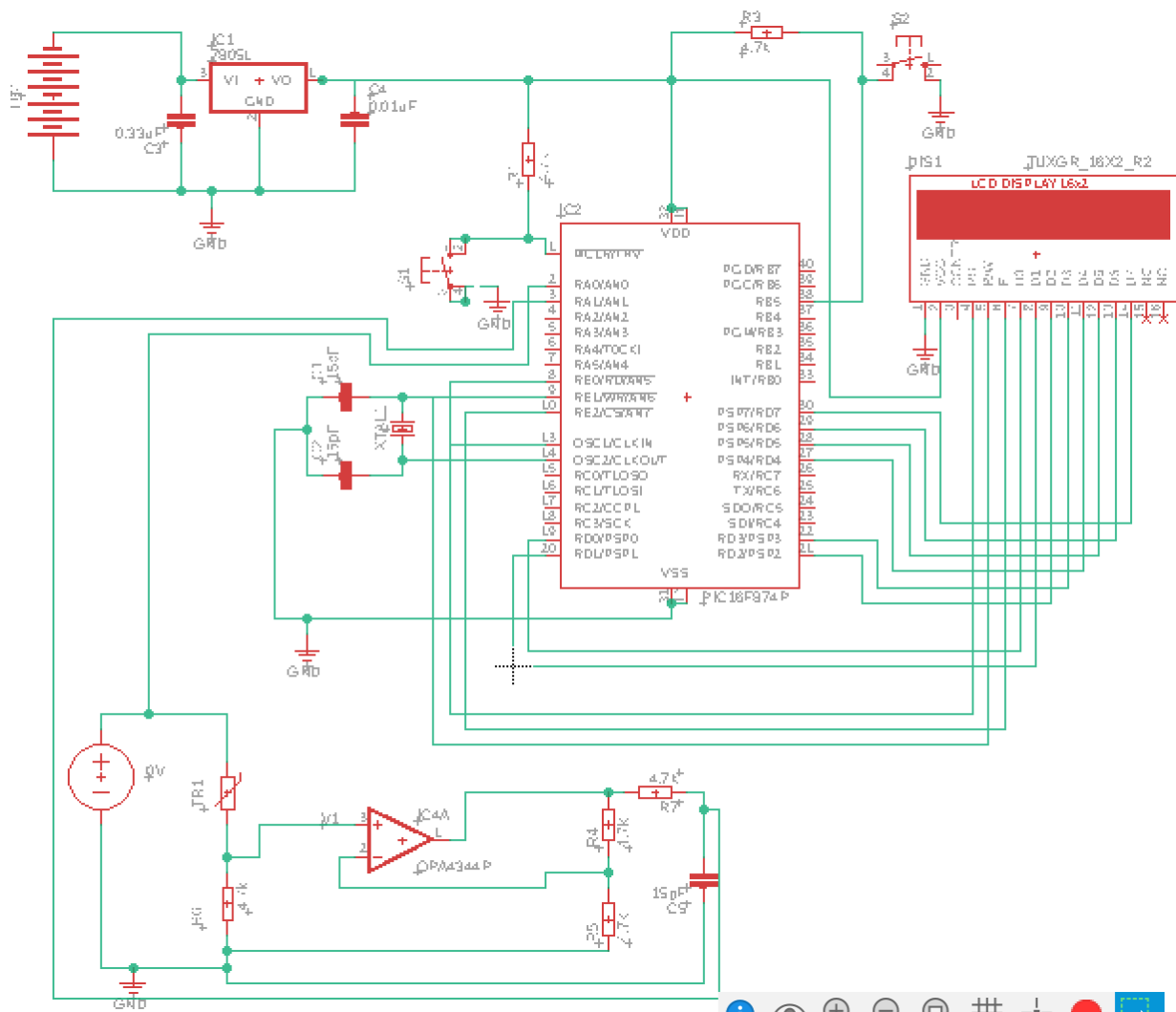
In a series circuit, the voltage drops across each resistor are proportional to its resistance (Playwithcircuit). If a resistor with a different value was used, then the voltage drop across each resistor will be different (Playwithcircuit). (Playwithcircuit) To put this into perspective, if a 10Ω resistor and a 100Ω resistor were placed in a series circuit, then 100Ω resistor will have a much larger voltage drop (Playwithcircuit).

In a parallel circuit, the total current flowing through the circuit is the sum of the currents flowing through each branch (Playwithcircuit). If resistors of different values in parallel circuits were used, then the current flowing through will be differ to each other, with more current flowing through the resistor with lower resistance (Playwithcircuit).

In a parallel circuit, the total current flowing through the circuit is the sum of the currents flowing through each branch_(electronics-tutorials). If the resistors of different values in parallel circuits were used, then the current flowing through the circuit will differ to each other, with more current flowing through the resistor with lower resistance (electronics-tutorials).

We connected our LCD display to the voltage regulator circuit and designed the switch to be independent from other circuits to ensure it controls the voltage supply by connecting the switch to the MCLR pin on the microcontroller which is used to reset the PIC or turn it on for the first time (electronics-tutorials).

We used an independent voltage source in our voltage circuit divider in our AFE circuit and a quad operational amplifier in our amplifier region of our circuit in the AFE, which follows the designated components we compiled in our circuit when we designed the AFE in MATLAB simulink (electronics-tutorials). We optimised for 15 picofarads for our capacitors in our oscillator circuit connected to our crystal clocked at 8 Mhz (electronics-tutorials).



input is read whilst condec is how the BCD conversion and scaling for the LCD display is carried out.

LCD 1 and 2 share multiple functions that we call such as:

- CLEAR - Resets the LCD Display
- SET_ADDR - Start a new line on the LCD Display
- TYPE_COM - Send any commands for the LCD Display to the CMD register
- TYPE_CHAR - moves any letter we want to send to the LCD display to its register
- SET_AN0 - changes the analogue channel to AN0
- SET_AN1 - changes the analogue channel to AN1
- DELAYS - There are three that cause delays of 15ms, 1s and 5s needed to have gaps in necessary parts of the program

The program will continue to loop through the main loop until RB0 changes values it is by default on LCD1 as RB0 is 0 then will switch to LCD2 when RB0 is 1. Both functions will call the shared functions except from AN0 and AN1

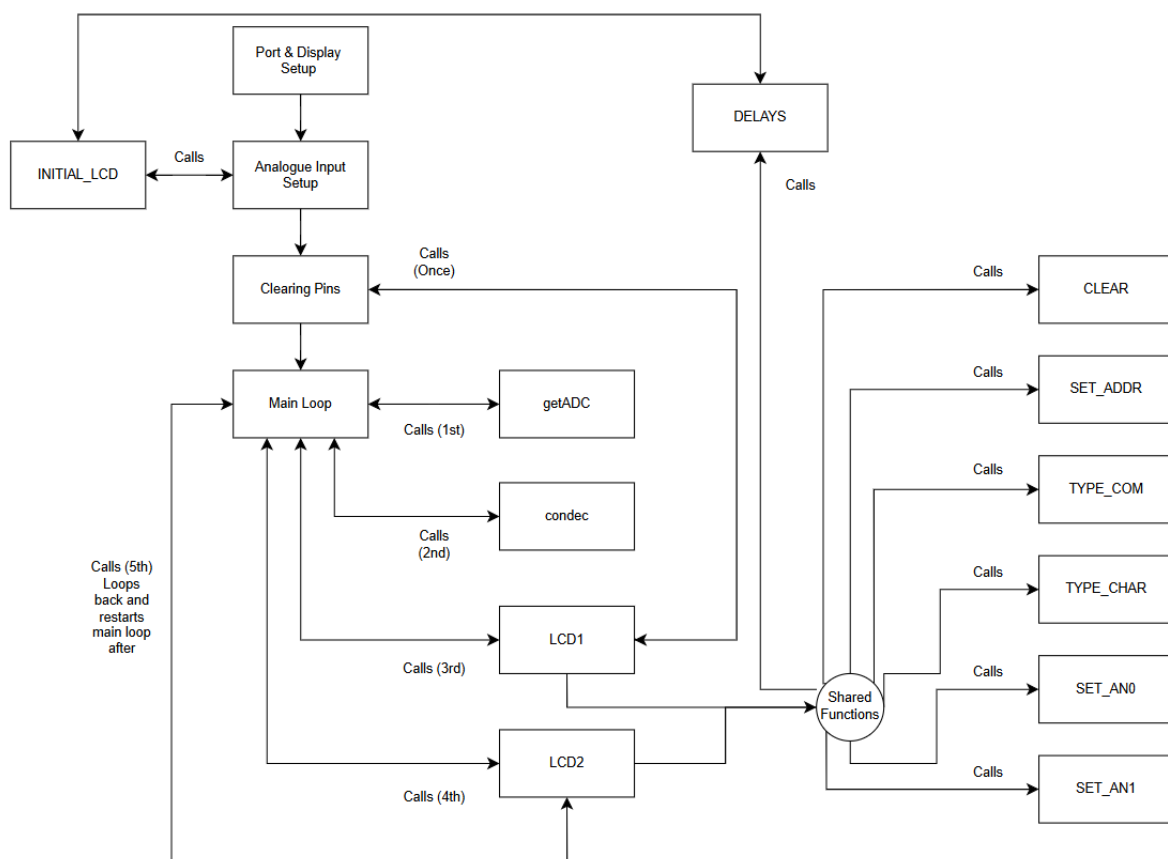


Figure 27: Program Structure

3.2 Firmware Configuration for PIC16F877A

In this project, the firmware configuration can be seen in the datasheet for PIC16F877A, we are using a 40 pin so there are only eight inputs (see Figure 30) for

the registers (ADCON0 and ADCON1) (Microchip, 2003). The register that we decided to use was ADCON0 for the LCD display and ADCON1 to set up the analogue input, the pins we had chosen to use are the AN0 and AN1 pin (Microchip, 2003).

At the start of the main program, we have a part of our code (see Figure 28) that configures the program. By going through Window > Target View Memory > Configuration Bits and on MPLABX you can see what settings (fields) have been turned off or on (See Figure 29). All the fields have been disabled, and the oscillator has been set to high speed.

```
; PIC16F877A Configuration Bit Settings: __config 0xFF3A
__CONFIG _FOSC_HS & _WDTE_OFF & _PWRTE_OFF & _BOREN_OFF & _LVP_OFF & _CPD_OFF & _WRT_OFF & _CP_OFF
```

Figure 28: Screenshot of Assembly Code changing the Configuration Bit Settings

Field	Option	Category	Setting
FOSC	HS	Oscillator Selection bits	HS oscillator
WDTE	OFF	Watchdog Timer Enable bit	WDT disabled
PWRTE	OFF	Power-up Timer Enable bit	PWRT disabled
BOREN	OFF	Brown-out Reset Enable bit	BOR disabled
LVP	OFF	Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit	RB3 is digital I/O, HV on MCLR must be used for programming
CPD	OFF	Data EEPROM Memory Code Protection bit	Data EEPROM code protection off
WRT	OFF	Flash Program Memory Write Enable bits	Write protection off; all program memory may be written to by EECON control
CP	OFF	Flash Program Memory Code Protection bit	Code protection off

Figure 29: Screenshot of the Configuration Bit Settings

11.0 ANALOG-TO-DIGITAL CONVERTER (A/D) MODULE

The Analog-to-Digital (A/D) Converter module has five inputs for the 28-pin devices and eight for the 40/44-pin devices.

The conversion of an analog input signal results in a corresponding 10-bit digital number. The A/D module has high and low-voltage reference input that is software selectable to some combination of V_{DD} , V_{SS} , RA2 or RA3.

The A/D converter has a unique feature of being able to operate while the device is in Sleep mode. To operate in Sleep, the A/D clock must be derived from the A/D's internal RC oscillator.

The A/D module has four registers. These registers are:

- A/D Result High Register (ADRESH)
- A/D Result Low Register (ADRESL)
- A/D Control Register 0 (ADCON0)
- A/D Control Register 1 (ADCON1)

The ADCON0 register, shown in Register 11-1, controls the operation of the A/D module. The ADCON1 register, shown in Register 11-2, configures the functions of the port pins. The port pins can be configured as analog inputs (RA3 can also be the voltage reference) or as digital I/O.

Additional information on using the A/D module can be found in the PIC® Mid-Range MCU Family Reference Manual (DS33023).

REGISTER 11-1: ADCON0 REGISTER (ADDRESS 1Fh)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON
bit 7							bit 0

bit 7-8 **ADCS1:ADCS0: A/D Conversion Clock Select bits (ADCON0 bits in bold)**

ADCON1 <ADC S2>	ADCON0 <ADC S1:ADC S0>	Clock Conversion
0	00	$F_{osc}/2$
0	01	$F_{osc}/8$
0	10	$F_{osc}/32$
0	11	FRC (clock derived from the internal A/D RC oscillator)
1	00	$F_{osc}/4$
1	01	$F_{osc}/16$
1	10	$F_{osc}/64$
1	11	FRC (clock derived from the internal A/D RC oscillator)

bit 5-3 **CHS2:CHS0: Analog Channel Select bits**

000 = Channel 0 (AN0)
 001 = Channel 1 (AN1)
 010 = Channel 2 (AN2)
 011 = Channel 3 (AN3)
 100 = Channel 4 (AN4)
 101 = Channel 5 (AN5)
 110 = Channel 6 (AN6)
 111 = Channel 7 (AN7)

Note: The PIC16F873A/876A devices only implement A/D channels 0 through 4; the unimplemented selections are reserved. Do not select any unimplemented channels with these devices.

bit 2 **GO/DONE: A/D Conversion Status bit**

When ADON = 1:

1 = A/D conversion in progress (setting this bit starts the A/D conversion which is automatically cleared by hardware when the A/D conversion is complete)
 0 = A/D conversion not in progress

bit 1 **Unimplemented: Read as '0'**

bit 0 **ADON: A/D On bit**

1 = A/D converter module is powered up
 0 = A/D converter module is shut-off and consumes no operating current

Legend:

R = Readable bit W = Writable bit U = Unimplemented bit, read as '0'
 - n = Value at POR '1' = Bit is set '0' = Bit is cleared x = Bit is unknown

Figure 30: Screenshot of the ADC section of the PIC16F877A Datasheet (Microchip, 2003)

3.3 LCD Driver & Timing

The LCD driver is a separate file from the main program that has global variables accessed by the main program. The initialisation of the LCD occurs here as well as the timers for the program, these functions are called by the main program.

Figure 31,32 and 33 shows the process of the LCD display being set up, before step 1 is a repeated process which is done before because the BF (busy flag) cannot be checked. Afterwards, step 1 sets the function by sending b '00111000' and step 2 turns the display off by sending b '00001000'. Step 3 turns off the display by sending b '00000001', this is repeated in the CLEAR function, which is a global function, this is reused so that we could clear the display whenever we need to. Step 4 is when entry mode is set by sending b '00000110' and step 5 turns the display back on by sending b '00001100'. If this process is not done before the main program the LCD display cannot be used, we used section 12 of the LCD datasheet for all the instructions (FORDATA, 2005).

Figure 32 shows the delays used in the main and secondary program so that there are delays between some instructions. As seen in Figure 25 each of the codes needed before BF can be checked need a delay between them so WAIT_15ms is used since all of them need delays > 15ms. The other delays (WAIT_1s uses WAIT_15ms and WAIT_5s uses WAIT_1s) use other delay functions and repeat it to make a larger delay, WAIT_1s does WAIT_15ms 67 times ($15 * 67 = 1005\text{ms}$ (1s)), WAIT_5s repeats WAIT_1s 5 times. These delays are used to cause pause between different display messages.

15. INITIALIZATION

15.1 8-bit interface mode (Condition: fosc = 270KHZ)

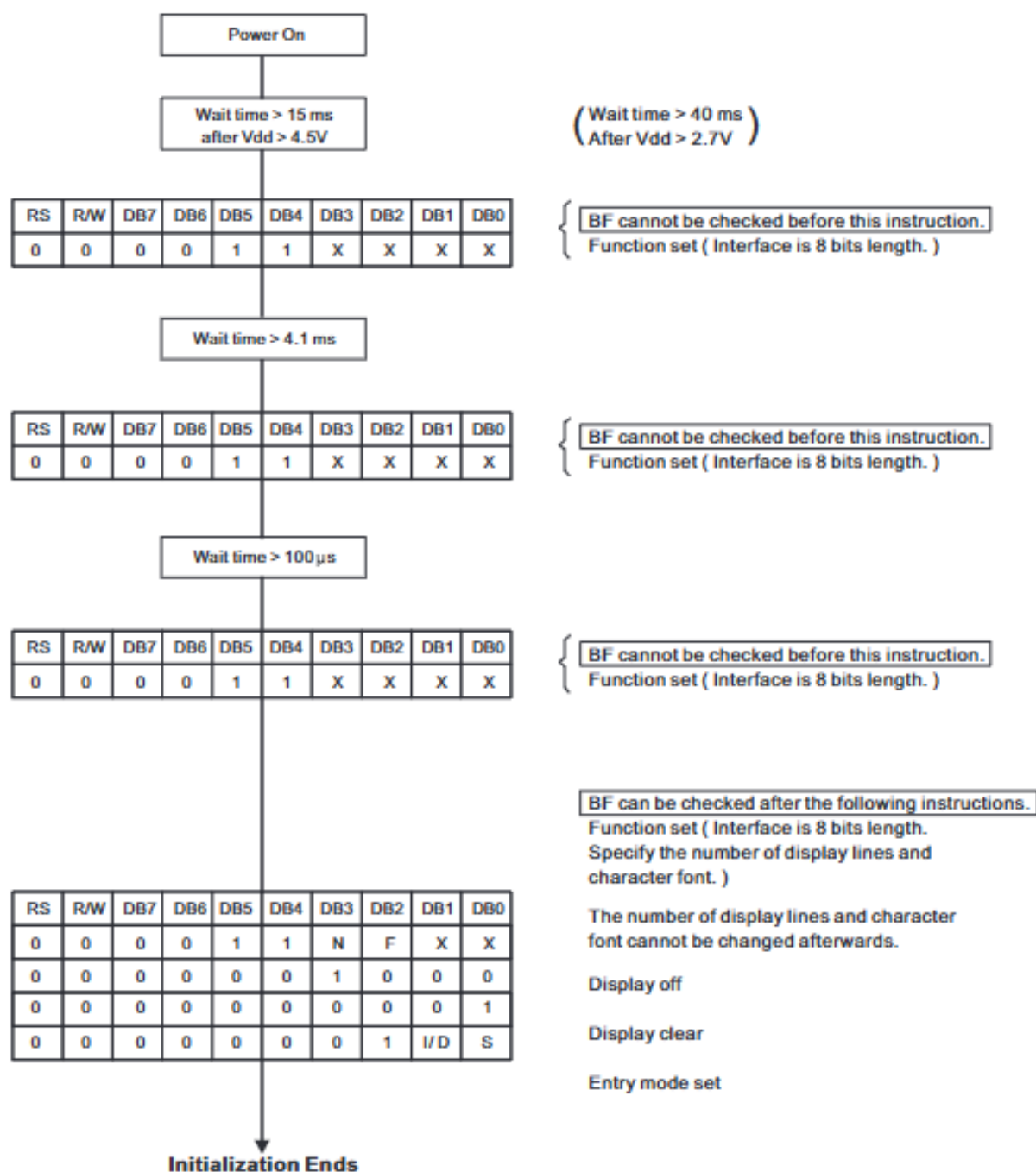


Figure 31: 8-bit initialisation (FORDATA, 2005)

code

INITIAL_LCD

```
BCF PORTE,1 ;E=0
BCF PORTE,2 ;RS=0
MOVLW b'00110000' ;D7-D0: 0011XXXX
BSF PORTE,1 ;E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ;E=0

;WAIT FOR 15ms
CALL WAIT_15ms

BCF PORTE,1 ;E=0
BCF PORTE,2 ;RS=0
MOVLW b'00110000' ;D7-D0: 0011XXXX
BSF PORTE,1 ;E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ;E=0

;WAIT FOR 15ms
CALL WAIT_15ms
BCF PORTE,1 ;E=0
BCF PORTE,2 ;RS=0
MOVLW b'00110000' ;D7-D0: 0011XXXX
BSF PORTE,1 ;E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ;E=0
```

Figure 32: First step of Setting up the LCD so BF can be read

```

; STEP 1 - FUNCTION SET (8 LINE)
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00111000' ; D7-D0: 00111XXX
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 15ms
BCF PORTE,1 ; E=0

; STEP 2 - DISPLAY OFF
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00001000' ; D7-D0: 00001000
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 500ns
BCF PORTE,1 ; E=0

; STEP 3 - DISPLAY CLEAR
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00000001' ; D7-D0: 00000001
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 500ns
BCF PORTE,1 ; E=0

; STEP 4 - ENTRY MODE SET
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00000110' ; D7-D0: 00000110
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; WAIT FOR 500ns
BCF PORTE,1 ; E=0
; STEP 5 - DISPLAY ON
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00001100' ; D7-D0: 00001111
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 500ns
BCF PORTE,1 ; E=0
CALL WAIT_15ms
GOTO FINISH

```

Figure 33: Second step of setting up LCD

```

WAIT_15ms
    MOV LW 0x57
    MOV WF counter_2

OUTER_LOOP15
    MOV WF counter_1
    DEC F counter_2,1
    BTF SC STATUS,Z
    GOTO FINISH

INNER_LOOP15
    DEC F counter_1,1
    BTF SS STATUS,Z
    GOTO INNER_LOOP15
    GOTO OUTER_LOOP15

```

Figure 34: Example of delays of the LCD Driver

3.4 Binary Coded Decimal (BCD) Conversion and Scaling

```

; Convert input to decimal .....
condec MOV WF ADbin ; get ADC result
      CLRF huns ; zero hundreds digit
      CLRF tens ; zero tens digit
      CLRF ones ; zero ones digit
      ; Calculate hundreds.....
      BSF STATUS,C ; set carry for subtract
      MOVLW D'100' ; load 100
sub1 SUBWF ADbin ; and subtract from result
      INCF huns ; count number of loops
      BTF SC STATUS,C ; and check if done
      GOTO sub1 ; no, carry on
      ADDWF ADbin ; yes, add 100 back on
      DEC F huns ; and correct loop count
      ; Calculate tens digit.....
      BSF STATUS,C ; repeat process for tens
      MOVLW D'10' ; load 10
sub2 SUBWF ADbin ; and subtract from result
      INCF tens ; count number of loops
      BTF SC STATUS,C ; and check if done
      GOTO sub2 ; no, carry on
      ADDWF ADbin ; yes, add 100 back on
      DEC F tens ; and correct loop count
      MOV F ADbin,W ; load remainder
      MOV WF ones ; and store as ones digit
      RETURN ; done

```

Figure 35: Screenshot of Assembly Code doing BCD Conversion and Scaling

BCD conversion and scaling is when each number is split into binary (ex. 375 -, 3 = 0011, 7 = 0111, 5 = 0101) so that it can be displayed on the LCD display and under the main file is where the conversion is taking place.

3.5 Input Selection

For our code we chose to use PORTB and ADCON0 as our inputs and in Figure 36 and 37 it is shown how when RB0 it will stay in the loop if it is a 0 then when the value is changed to a 1 it will break out of the loop and change the channel. The other two functions (SET_AN0 and SET_AN1) show ADCON0 being updated with a value which will be used when simulating this code in PICSIM. These two functions will allow for each thermistor to be simulated separately since different pins will be set. In Figure 30, it is shown what bit CHS2 to CHS0 need to be to decide what channel is being selected.

```

SET_AN0
    MOVLW    B'01000001'
    MOVWF    ADCON0
    RETURN

SET_AN1
    MOVLW    B'01001001'
    MOVWF    ADCON0
    RETURN

```

Figure 36: Screenshot of the channels being changed selected

```

BTFSS    PORTB,RB0
CALL     CLEAR
BTFSS    PORTB,RB0
CALL     SET_AN0

```

Figure 37: Screenshot of RB0 value deciding if channel is changed

Chapter 4: Result & Testing

4.1 Behavioural Simulations of AFE

In MATLAB/Simulink, we connected the circuit properly and in the amplifier module, a combination of Op-Amp, feedback resistance and input resistance were used to form a non-inverting amplifier. In the low-pass filter circuit module, a simple RC filter is used for connection by using the formula $G=1+R_f/R_i$ to adjust the size of the resistance and then use the formula $f_c=1/2\pi RC$ to adjust the sizes of the resistance and capacitance in the filter. When resistance becomes larger a clearer waveform will show up and increasing the capacitance the amplitude of the image will shrink. Finally, we connected V_{out} to the positive terminal of the voltage sensor and the negative terminal is connected to the electrical reference, and the Simscape Physical Signal from the voltage sensor is connected to the scope via the PS-Simulink

Converter to show the filtered image. This will ensure that there is only one electrical reference in the entire circuit.

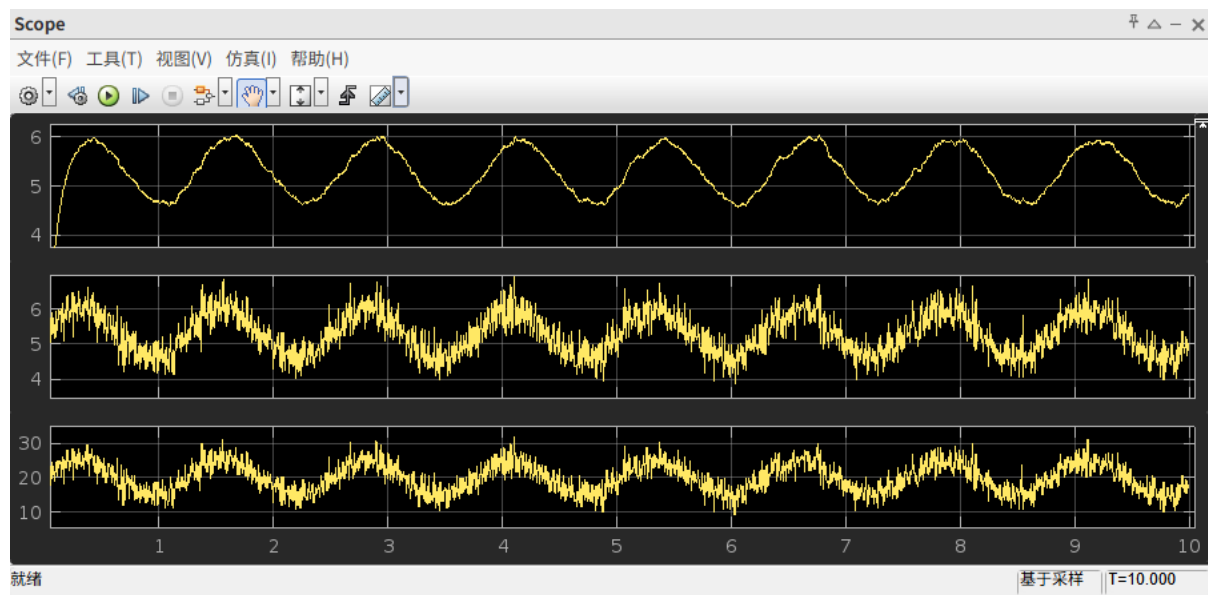


Figure 38: Image of Behaviour Simulation on MATLAB

4.2 Testing Methodology and Test Vectors Used

For the testing stage, we used MATLAB to simulate the AFE circuit, this gave us an understanding of what a change in resistance and capacitance will do to the AFE circuit. This is because we chose to use an RC filter circuit as our lowpass filter.

For the assembly code in MPLABX we used PICSIM and the tools of MATLAB to simulate so we could see the results of the code before any hardware is produced, then in MPLABX we used breakpoint to stop the program whenever we needed. Afterwards, we used tools like the stopwatch to see how long our delays last (see Figure 39), used Target Memory Views to see SFRs like ADCON0 and ADCON1 (see Figure 40 and 41), and used the I/O pins to change the value of RB0 (see Figure 42 and 43).

The reason for these tests were to see whether our circuits or our software would work before making any circuits and setting up the motherboard. In addition to this it let us understand if there were any errors occurring or improvements to make.

4.3 Use of MPLABX simulations for testing and results

```
246      inid      ; initialise the LCD
247      CALL     WAIT_15ms
248      CALL     INITIAL_LCD
249      RETURN
250
251      SET_AN0
252      MOVLW    B'01000001'
253      MOVWF    ADCON0
254      RETURN
255
256      SET_AN1
257      MOVLW    B'01001001'
258      MOVWF    ADCON0
259      RETURN
260
261      END
262
```

Stopwatch

- Target halted. Stopwatch cycle count = 465 (465 μ s)
- Target halted. Stopwatch cycle count = 467 (467 μ s)
- Target halted. Stopwatch cycle count = 468 (468 μ s)
- Target halted. Stopwatch cycle count = 469 (469 μ s)
- Target halted. Stopwatch cycle count = 471 (471 μ s)
- Target halted. Stopwatch cycle count = 472 (472 μ s)
- Target halted. Stopwatch cycle count = 473 (473 μ s)
- Target halted. Stopwatch cycle count = 475 (475 μ s)
- Target halted. Stopwatch cycle count = 29913 (29.913 ms)

Figure 39: This shows a delay function used the program

Figure 39 has a breakpoint when the function is called and after so that we can record that delay will happen when the function is called.

	org	0x00		; Default start address	
	NOP			; required for ICD mode	
				; Port & display setup.....	
	BANKSEL	TRISC		; Select bank 1	
	CLRF	TRISD		; Display port is output	
	movlw	0x0f			
	movwf	TRISA			
	CLRF	TRISE		; Display port is output	
	MOVLW	B'00000010'		; Analogue input setup code	
	MOVWF	ADCON1		; Left justify result,	
				; Port A = analogue inputs	
	BANKSEL	PORTC		; Select bank 0	
	CLRF	PORTD		; Clear display outputs	

ess /	Name	Hex	Decimal	Binary	Char
	CCPR2H	0x00	0	00000000	'.'
	CCP2CON	0x00	0	00000000	'.'
	ADRESH	0x00	0	00000000	'.'
	ADCON0	0x00	0	00000000	'.'
	OPTION_REG	0xFF	255	11111111	'ÿ'
	TRISA	0x3F	63	00111111	'?'
	TRISB	0xFF	255	11111111	'ÿ'
	TRISC	0xFF	255	11111111	'ÿ'
	TRISD	0xFF	255	11111111	'ÿ'
	TRISE	0x07	7	00000111	'.'
	PIE1	0x00	0	00000000	'.'
	PIE2	0x00	0	00000000	'.'
	PCON	0x00	0	00000000	'.'
	SSPCON2	0x00	0	00000000	'.'
	PR2	0xFF	255	11111111	'ÿ'
	SSPAD	0x00	0	00000000	'.'
	SSPSTAT	0x00	0	00000000	'.'
	TXSTA	0x00	0	00000000	'.'
	SPBRG	0x00	0	00000000	'.'
	CMCON	0x07	7	00000111	'.'
	CVRCON	0x00	0	00000000	'.'
	ADRESL	0x00	0	00000000	'.'
	ADCON1	0x00	0	00000000	'.'
	EEDATA	0x00	0	00000000	'.'
	EEADR	0x00	0	00000000	'.'

Figure 40: ADCON0 set to 0

```

; Port & display setup.....
BANKSEL TRISC      ; Select bank 1
CLRF   TRISD      ; Display port is output
movlw  0x0f
movwf  TRISA
CLRF   TRISE      ; Display port is output
MOVLW  B'00000010' ; Analogue input setup code
MOVWF  ADCON1     ; Left justify result,
; Port A = analogue inputs
BANKSEL PORTC      ; Select bank 0
CLRF   PORTD      ; Clear display outputs
CLRF   PORTE      ; Clear display outputs
BTFSS  PORTB,RB1
MOVLW  B'01000001' ; Analogue input setup code
MOVWF  ADCON0     ; f/8, RA0, done, enable
CALL   inid       ; Initialise the display
; Clearing Pins
BCF    PORTB,RB0

```

ss /	Name	Hex	Decimal	Binary	Char
	CCPR2H	0x00	0	00000000	'.'
	CCP2CON	0x00	0	00000000	'.'
	ADRESH	0x00	0	00000000	'.'
	ADCON0	0x41	65	01000001	'A'
	OPTION_REG	0xFF	255	11111111	'ÿ'
	TRISA	0x0F	15	00001111	'.'
	TRISB	0xFF	255	11111111	'ÿ'
	TRISC	0xFF	255	11111111	'ÿ'
	TRISD	0x00	0	00000000	'.'
	TRISE	0x00	0	00000000	'.'
	PIE1	0x00	0	00000000	'.'
	PIE2	0x00	0	00000000	'.'
	PCON	0x00	0	00000000	'.'
	SSPCON2	0x00	0	00000000	'.'
	PR2	0xFF	255	11111111	'ÿ'
	SSPAD	0x00	0	00000000	'.'
	SSPSTAT	0x00	0	00000000	'.'
	TXSTA	0x00	0	00000000	'.'
	SPBRG	0x00	0	00000000	'.'
	CMCON	0x07	7	00000111	'.'
	CVRCON	0x00	0	00000000	'.'
	ADRESL	0x00	0	00000000	'.'
	ADCON1	0x00	0	00000000	'.'

Figure 41: ADCON being changed to 0x47

Figures 40 and 41 show a test being carried out by setting up a breakpoint before the port display setup and after the analogue input starts which shows ADCON0 changing from 0 to 0x47. This was to check if the channel was being updated from AN0 to AN1.

```

73      CALL    LCD1      ; display input
74      GOTO    start     ; jump to main loop
75      ; SUBROUTINES ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
76      ; Read ADC input and store ;;;;;;;;;;;;;;;;;;;;;;;;;
77      getADC BSF      ADCON0,GO ; start ADC..
78      wait  BTFSC    ADCON0,GO ; ..and wait for finish
79      GOTO    wait
80      MOVF    ADRESH,W ; store result high byte
81      RETURN
82      ; Convert input to decimal ;;;;;;;;;;;;;;;;;;;;;;;;;
83      condec MOVWF    ADbIn ; get ADC result
84      CLRF    huns ; zero hundreds digit
85      CLRF    tens ; zero tens digit
86      CLRF    ones ; zero ones digit
87      ; Calculate hundreds ;;;;;;;;;;;;;;;;;;;;;;;;;
88      BSF     STATUS,C ; set carry for subtract
89      MOVLW   D'100' ; load 100
90      sub1    SUBWF    ADbIn ; and subtract from result
91      INCF    huns ; count number of loops
92      BTFSC   STATUS,C ; and check if done
93      GOTO    sub1 ; no, carry on
94      ADDWF    ADbIn ; yes, add 100 back on
95      DECF    huns ; and correct loop count
96      ; Calculate tens digit ;;;;;;;;;;;;;;;;;;;;;;;;;
97      BSF     STATUS,C ; repeat process for tens
98      MOVLW   D'10' ; load 10
99      sub2    SUBWF    ADbIn ; and subtract from result
100     INCF    tens ; count number of loops
101     BTFSC   STATUS,C ; and check if done
102     GOTO    sub2 ; no, carry on
103     ADDWF    ADbIn ; yes, add 100 back on
104     DECF    tens ; and correct loop count
105     MOVF    ADbIn,W ; load remainder
106     MOVWF    ones ; and store as ones digit
107     RETURN ; done
108
109     LCD1
110     BTFSS    PORTB,RB0
111     CALL     SET_AN0
112     RETURN
113     ; Output to display ;;;;;;;;;;;;;;;;;;;;;;;;;
114     MOVWF    000 ; code to home cursor

```

Pin	Mode	Value
RB0	Din	0

Figure 42: The result of the main loop if RB0 is 0

```

182
183     LCD2
184     BTFSS    PORTB,RB0
185     CALL     SET_AN1
186     ; Output to display ;;;;;;;;;;;;;;;;;;;;;;;;;
187     MOVWF    000 ; code to home cursor
188     MOVWF    cmd_reg
189     CALL     TYPE_COM ; output it to display
190     ; Convert digits to ASCII and display ;;;;;;;;;;;;;;;;;
191     MOVLW    030 ; load ASCII offset
192     ADDWF    huns ; convert hundreds to ASCII
193     ADDWF    tens ; convert tens to ASCII
194     ADDWF    ones ; convert ones to ASCII
195     MOVF    huns,W ; load hundreds code
196     MOVWF    lcd_reg
197     CALL     TYPE_CHAR ; and send to display
198     MOVLW    '.' ; load point code
199     MOVWF    lcd_reg
200     CALL     TYPE_CHAR ; and output

```

Pin	Mode	Value
RB0	Din	1

Figure 43: The function entered during the main loop if RB0 is 1

Figures 42 and 43 show how the main will call LCD 2 when RB0 is changed to 1, this was tested to see what would happen later when we clicked our electronic switch.

4.4 Virtualisation of solution on PICSIM and results



Figure 44: Function LCD1 running channel AN0



Figure 45: Function LCD2 switched to by RB0, running channel AN1

Figure 44 and 45 shows the changes in the LCD display when RB0 is clicked as the message will change as the channel. Furthermore, the figures show a change in

voltage on the LCD display when we change the inputs on the channels using PICSIM.

Conclusion

Overall, we successfully managed to produce software useable by our microcontroller as well as designing schematics for our hardware through tools such as MATLAB, MPLABX and Fusion 360. The software created has managed to collect data from the AFE circuit and put it into a readable format that can be displayed on an LCD display after its display setup. In addition to this, it has been programmed to be controlled through one electronic switch which will alternate between the between the channels of the ADC which enables the LCD display to show the results of two sensors.

Our independent research and testing have coherently made this possible as we discovered how we wish to design this project as well as how to fix errors and make improvements. The testing of our software through PICSIM and MPLABX helped up decide the logic of our system and how the program will be structured. These choices made a positive impact on how we wanted our main loop to work and what functions it needed to call.

Moreover, our research allowed us to learn what the layout of circuitry to look at as we researched what data acquisition systems are. This allowed us to select what electronic switches we would use, what amplifier and filter, what pins we were using etc. Software like MATLAB allowed us to run behavioural simulations of the AFE circuit so we knew what signal we were collected then we adjust other factors to achieve the simulation results we wanted.

References

Park, J. and Mackay, S. (2003) *Practical data acquisition for instrumentation and control systems* Newnes, pp. 1–2.

Morris, A.S. (2001) *Measurement and Instrumentation Principles* Chantilly: Elsevier Science & Technology.

Kyle Forinash and Wolfgang Christian *Sound: An Interactive Book*, pp. 2.1.2.

Plonus, M. (2020) 9 - *Digital systems*, in Plonus, M. (ed.) Butterworth-Heinemann, pp. 355–480.

RC Filter. Available at: <https://www.electronics-tutorials.ws/filter>

Dewesoft (2023) *Filters*. Available at: <https://training.dewesoft.com/filters>

RS Components (2024) *A Complete Guide to Resistors*. Available at: <https://uk.rs-online.com/web/content/discovery/ideas-and-advice/resistors-guide>

Horowitz, P. and Hill, W. (2015) *The art of electronics* New York: Cambridge University Press.

RS Components *Toshin Kogyo 100 μ F Electrolytic Capacitor*. Available at: <https://uk.rs-online.com/web/p/aluminium-capacitors/1443373>

Jung, W. (2004) *Op Amp Applications Handbook*. 1st edn. Chantilly: Elsevier Science & Technology.

Ravi Teja (2024) *Switches*. Available at: <https://www.electronicshub.org/switches/>

MOSFET (2024) Available at: <https://uk.rs-online.com/mosfet-guide>

Arrow (2023) *Basics of Analog to Digital Converters*. Available at: <https://www.arrow.com/en/research-and-events/articles/engineering-resource-basics-of-analog-to-digital-converters>

dwyeromega *Thermistor*. Available at: <https://www.dwyeromega.com/en-us/resources/thermistor>

Rs-Online *EPCOS Thermistor, 10 Ω* . Available at: <https://uk.rs-online.com/web/p/thermistor>

The MathWorks. Inc (2025a) What is MATLAB. Available at: <https://uk.mathworks.com/discovery/what-is-matlab.html>

The MathWorks. Inc (2025h) Product Description. Available at: https://uk.mathworks.com/help/matlab/learn_matlab/product-description.html

Microchip (2024) *Software Tools*. Available at: <https://developerhelp.microchip.com/xwiki/bin/view/software-tools/ides/x/>

Luis Gambôa (2024) 'PICSimLab 0.9.2', Available at: https://lcgamboa.github.io/picsimlab_docs/stable/picsimlab.pdf.

Medium (2024) *Understanding Embedded System Clocks and Timers*. Available at: <https://medium.com/understanding-embedded-system-clocks-and-timers>

FORDATA (2005) *LCD Datasheet*. Available at: <https://www.farnell.com/datasheets/653650.pdf>

Quest Components (2019) *How the Wrong Resistor Can Ruin a Project*. Available at: <https://www.questcomp.com/blog/how-the-wrong-resistor-can-ruin-a-project>

playwithcircuit *What is a Resistor? Working and Types of Resistor*. Available at: <https://playwithcircuit.com/what-is-a-resistor-working-and-types-of-resistor/>

electronics-tutorials *Resistors in Parallel*. Available at: https://www.electronics-tutorials.ws/resistor/res_4.html

Microchip (2003) *PIC16F877A Datasheet*. Available at: <https://ww1.microchip.com/downloads/en/devicedoc/39582b.pdf>

Monolithic Power *Voltage Regulator*. Available

at: <https://www.monolithicpower.com/en/learning/resources/voltage-regulator-types>

Avnet (2023) *Crucial role of analog front ends, creating bridge to digital*

world. Available at: <https://www.avnet.com/article/crucial-role-of-analog-front-ends-creating-bridge-to-digital-world/>

Aswinth Raj (2015) '16x2 LCD Module', Available

at: <https://circuitdigest.com/article/16x2-lcd-display-module-pinout-datasheet>

Appendices

A

AFE

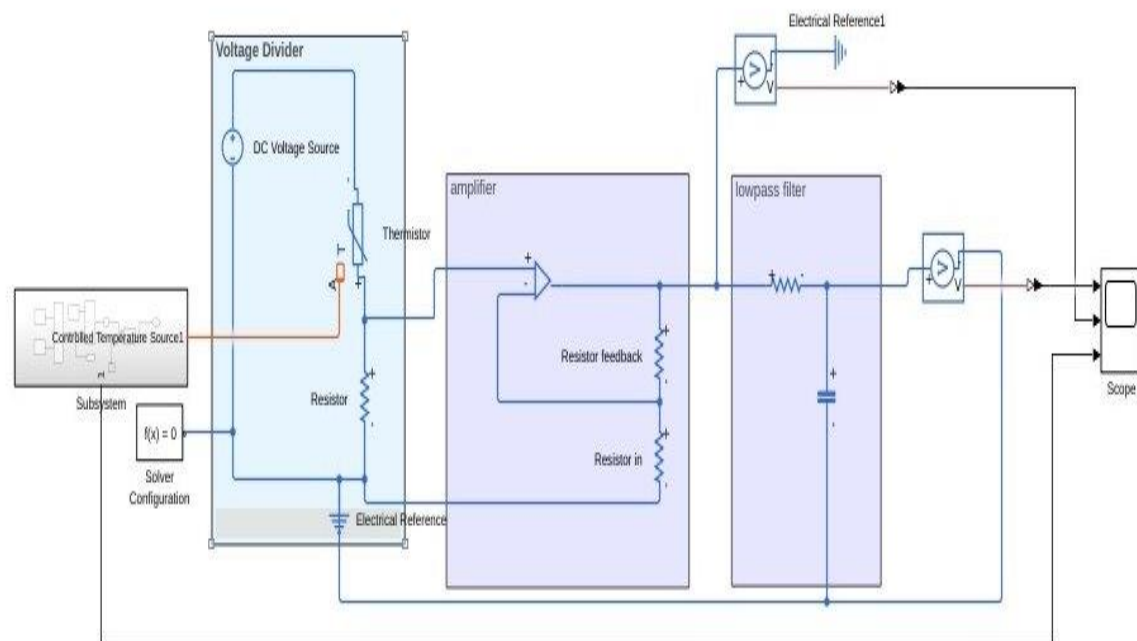


Figure 46: A schematic of the AFE (Analogue Front-End) Circuit

PCB Layout



B Driver

```

1  #include "P16f877A.INC"
2  global INITIAL_LCD
3  global WAIT_15ms
4  global WAIT_1s
5  global WAIT_5s
6  global TYPE_CHAR
7  global TYPE_COM
8  global SET_ADDR
9  global CLEAR
10
11  counter_1    equ 0x20
12  counter_2    equ 0x21
13  counter_3    equ 0x22
14  counter_4    equ 0x23
15  counter_5    equ 0x24
16  counter_6    equ 0x25
17  lcd_reg      EQU 35 ; LCD register to hold char or command information
18  cmd_reg      EQU 36 ; LCD register to hold char or command information
19
20  code
21  INITIAL_LCD BCF PORTE,1 ;E=0
22              BCF PORTE,2 ;RS=0
23              MOVLW b'00110000' ;D7-D0: 0011XXXX
24              BSF PORTE,1 ;E=1
25              NOP
26              MOVWF PORTD
27              NOP ;WAIT FOR 500ns
28              BCF PORTE,1 ;E=0
29
30              ;WAIT FOR 15ms
31              CALL WAIT_15ms
32
33              BCF PORTE,1 ;E=0
34              BCF PORTE,2 ;RS=0
35              MOVLW b'00110000' ;D7-D0: 0011XXXX
36              BSF PORTE,1 ;E=1
37              NOP
38              MOVWF PORTD
39              NOP ;WAIT FOR 500ns
40              BCF PORTE,1 ;E=0
41
42              ;WAIT FOR 15ms
43              CALL WAIT_15ms
44              BCF PORTE,1 ;E=0
45              BCF PORTE,2 ;RS=0
46              MOVLW b'00110000' ;D7-D0: 0011XXXX
47              BSF PORTE,1 ;E=1
48              NOP
49              MOVWF PORTD
50              NOP ;WAIT FOR 500ns
51              BCF PORTE,1 ;E=0
52              ; STEP1 - FUNCTION SET (8 LINE)
53              CALL WAIT_15ms
54              BCF PORTE,1 ;E=0
55              BCF PORTE,2 ;RS=0

```

Figure 48: A listing of the code of the initialisation of the LCD Display

```

56      MOVLW b'00111000' ;D7-D0: 00111XXX
57      BSF PORTE,1 ;E=1
58      NOP
59      MOVWF PORTD
60      NOP ;WAIT FOR 500ns
61      BCF PORTE,1 ;E=0
62      ; STEP2 - DISPLAY OFF
63      CALL WAIT_15ms
64      BCF PORTE,1 ;E=0
65      BCF PORTE,2 ;RS=0
66      MOVLW b'00001000' ;D7-D0: 00001000
67      BSF PORTE,1 ;E=1
68      NOP
69      MOVWF PORTD
70      NOP ;WAIT FOR 500ns
71      BCF PORTE,1 ;E=0
72      ; STEP3 - DISPLAY CLEAR
73      CALL WAIT_15ms
74      BCF PORTE,1 ;E=0
75      BCF PORTE,2 ;RS=0
76      MOVLW b'00000001' ;D7-D0: 00000001
77      BSF PORTE,1 ;E=1
78      NOP
79      MOVWF PORTD
80      NOP ;WAIT FOR 500ns
81      BCF PORTE,1 ;E=0
82      ; STEP4 - ENTRY MODE SET
83      CALL WAIT_15ms
84      BCF PORTE,1 ;E=0
85      BCF PORTE,2 ;RS=0
86      MOVLW b'00000110' ;D7-D0: 00000110
87      BSF PORTE,1 ;E=1
88      NOP
89      MOVWF PORTD
90      NOP ;WAIT FOR 500ns
91      BCF PORTE,1 ;E=0
92      ; STEPS - DISPLAY ON
93      CALL WAIT_15ms
94      BCF PORTE,1 ;E=0
95      BCF PORTE,2 ;RS=0
96      MOVLW b'00001100' ;D7-D0: 00001111
97      BSF PORTE,1 ;E=1
98      NOP
99      MOVWF PORTD
100     NOP ;WAIT FOR 500ns
101     BCF PORTE,1 ;E=0
102     CALL WAIT_15ms
103     GOTO FINISH
104
105     TYPE_COM ;WAIT FOR 15ms
106     CALL WAIT_15ms
107     MOVF cmd_reg,0
108     BCF PORTE,1 ;E=0
109     BCF PORTE,2 ;RS=0
110     BSF PORTE,1 ;E=1

```

Figure 49: A listing of the code of the initialisation of the LCD Display and the function of the lcd_reg

```

111      NOP
112      MOVWF PORTD
113      NOP ;WAIT FOR 500ns
114      BCF PORTE,1 ;E=0
115      GOTO FINISH
116
117  TYPE_CHAR  CALL WAIT_15ms
118              MOVF lcd_reg,0
119              BCF PORTE,1 ;E=0
120              BSF PORTE,2 ;RS=1
121              BSF PORTE,1 ;E=1
122              MOVWF PORTD
123              NOP ;WAIT FOR 500ns
124              BCF PORTE,1 ;E=0
125              GOTO FINISH
126
127  ; SET DDRAM ADDRESS FOR 2ND ROW
128  SET_ADDR   CALL WAIT_15ms
129              BCF PORTE,1 ;E=0
130              BCF PORTE,2 ;RS=0
131              MOVLW b'11000000' ;Set address to 40h
132              BSF PORTE,1 ;E=1
133              MOVWF PORTD
134              NOP ;WAIT FOR 500ns
135              BCF PORTE,1 ;E=0
136              GOTO FINISH
137
138  CLEAR      CALL WAIT_15ms
139              BCF PORTE,1 ;E=0
140              BCF PORTE,2 ;RS=0
141              MOVLW b'00000001' ; clear display
142              BSF PORTE,1 ;E=1
143              MOVWF PORTD
144              NOP ;WAIT FOR 500ns
145              BCF PORTE,1 ;E=0
146              GOTO FINISH
147
148  WAIT_15ms
149              MOVLW 0x57
150              MOVWF counter_2
151
152  OUTER_LOOP15
153              MOVWF counter_1
154              DECF counter_2,1
155              BTFSC STATUS,Z
156              GOTO FINISH
157
158  INNER_LOOP15
159              DECF counter_1,1
160              BTFSS STATUS,Z
161              GOTO INNER_LOOP15
162              GOTO OUTER_LOOP15
163

```

Figure 50: A listing of the code of the function of the cmd_reg, clearing of the LCD Display and 15ms Delay

```

164      WAIT_1s
165      |      |      |      MOVLW 0x67
166      |      |      |      MOVWF counter_4
167
168      OUTER_LOOP1 MOVWF counter_3
169      |      |      |      DECF counter_4,1
170      |      |      |      BTFSC STATUS,2
171      |      |      |      GOTO FINISH
172
173      INNER_LOOP1 DECF counter_3,1
174      |      |      |      BTFSS STATUS,2
175      |      |      |      GOTO INNER_LOOP1
176      |      |      |      GOTO OUTER_LOOP1
177
178      WAIT_5s
179      |      |      |      MOVLW 0x5
180      |      |      |      MOVWF counter_6
181
182      OUTER_LOOPS
183      |      |      |      MOVWF counter_6
184      |      |      |      DECF counter_2,1
185      |      |      |      BTFSC STATUS,2
186      |      |      |      GOTO FINISH
187
188      INNER_LOOPS
189      |      |      |      DECF counter_5,1
190      |      |      |      BTFSS STATUS,2
191      |      |      |      GOTO INNER_LOOP1
192      |      |      |      GOTO OUTER_LOOP1
193
194      FINISH
195      |      |      |      NOP
196      |      |      |      RETURN
197

```

Figure 51: A listing of the code of the 1s and 5s Delay

Main LCD Display

```

1      list      p=16f877A      ; list directive to define processor
2      #include  <pl6f877A.inc> ; processor specific variable definitions
3
4      ; PIC16F877A Configuration Bit Settings: __config 0xFF3A
5      __CONFIG _FOSC_HS & _WDTE_OFF & _PWRTE_OFF & _BOREN_OFF & _LVP_OFF & _CPD_OFF & _WRT_OFF & _CP_OFF
6
7      ; #####
8      ; Declare variables defined elsewhere
9      extern INITIAL_LCD
10     extern WAIT_15ms
11     extern WAIT_1s
12     extern WAIT_5s
13     extern TYPE_CHAR
14     extern TYPE_COM
15     extern SET_ADDR
16     extern CLEAR
17
18     ; #####
19     ; Label declarations
20     ; note all locations in page 0 after 0x0b available as general RAM locations
21
22     ; GPR 70 - 75 allocated to included LCD display routine
23     count EQU 30      ; Counter for ADC setup delay
24     ADbin EQU 31      ; Binary input value
25     hunz EQU 32      ; Hundreds digit in decimal value
26     tens EQU 33      ; Tens digit in decimal value
27     ones EQU 34      ; Ones digit in decimal value
28     lcd_reg EQU 35    ; LCD register to hold char or command information
29     cmd_reg EQU 36    ; LCD register to hold char or command information
30
31     ; PROGRAM BEGINS ;#####
32     org 0x00          ; Default start address
33     NOP ; required for ICD mode
34     ; Port & display setup.....
35     BANKSEL TRISC      ; Select bank 1
36     CLRF TRISD         ; Display port is output
37     movlw 0x0f
38     movwf TRISA
39     CLRF TRISE         ; Display port is output
40     MOVLW B'00000010' ; Analogue input setup code
41     MOVWF ADCON1       ; Left justify result,
42     ; Port A = analogue inputs
43     BANKSEL PORTC      ; Select bank 0
44     CLRF PORTD         ; Clear display outputs
45     CLRF PORTE         ; Clear display outputs
46     BTFSS PORTB,RB1
47     MOVLW B'01000001' ; Analogue input setup code
48     MOVWF ADCON0       ; f/8, RA0, done, enable
49     CALL inid          ; Initialise the display

```

Figure 52: A listing of the code of the start of the LCD Display Program

```

50      ; Clearing Pins
51      BCF      PORTB,RB0
52      BCF      PORTB,RB4
53      BCF      PORTB,RB5
54      CALL     LCD1
55      ; MAIN LOOP ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
56      start    CALL     getADC      ; read input
57              CALL     condec      ; convert to decimal
58              BTFSC    PORTB,RB0
59              CALL     LCD2
60              BTFSC    PORTB,RB0
61              CALL     SET_AN1
62              BTFSS    PORTB,RB0
63              CALL     LCD1        ; display input
64              GOTO     start        ; jump to main loop
65      ; SUBROUTINES ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
66      ; Read ADC input and store .....
67      getADC    BSF      ADCON0,GO   ; start ADC..
68      wait      BTFSC    ADCON0,GO   ; ..and wait for finish
69              GOTO     wait
70              MOVF     ADRESH,W      ; store result high byte
71              RETURN
72      ; Convert input to decimal .....
73      condec    MOVWF    ADbin       ; get ADC result
74              CLRF     huns         ; zero hundreds digit
75              CLRF     tens         ; zero tens digit
76              CLRF     ones         ; zero ones digit
77      ; Calculate hundreds.....
78              BSF      STATUS,C      ; set carry for subtract
79              MOVLW    D'100'        ; load 100
80      subl      SUBWF    ADbin       ; and subtract from result
81              INCF     huns         ; count number of loops
82              BTFSC    STATUS,C      ; and check if done
83              GOTO     subl          ; no, carry on
84              ADDWF    ADbin         ; yes, add 100 back on
85              DECF     huns         ; and correct loop count
86      ; Calculate tens digit.....
87              BSF      STATUS,C      ; repeat process for tens
88              MOVLW    D'10'         ; load 10
89      sub2      SUBWF    ADbin       ; and subtract from result
90              INCF     tens         ; count number of loops
91              BTFSC    STATUS,C      ; and check if done
92              GOTO     sub2          ; no, carry on
93              ADDWF    ADbin         ; yes, add 100 back on
94              DECF     tens         ; and correct loop count
95              MOVF     ADbin,W       ; load remainder
96              MOVWF    ones         ; and store as ones digit
97              RETURN                ; done
98

```

Figure 53: A listing of the code of the Main Loop, ADC reading of the input and the BCD conversion and scaling

```

99      LCD1
100      BTFSS    PORTB,RB0
101      CALL     SET_ANO
102      ; Output to display.....
103      MOVLW    080          ; code to home cursor
104      MOVWF    cmd_reg
105      CALL     TYPE_COM      ; output it to display
106      ; Convert digits to ASCII and display.....
107      MOVLW    030          ; load ASCII offset
108      ADDWF    huns          ; convert hundreds to ASCII
109      ADDWF    tens          ; convert tens to ASCII
110      ADDWF    ones          ; convert ones to ASCII
111      MOVF     huns,W        ; load hundreds code
112      MOVWF    lcd_reg
113      CALL     TYPE_CHAR     ; and send to display
114      MOVLW    '.'          ; load point code
115      MOVWF    lcd_reg
116      CALL     TYPE_CHAR     ; and output
117      MOVF     tens,W        ; load tens code
118      MOVWF    lcd_reg
119      CALL     TYPE_CHAR     ; and output
120      MOVF     ones,W        ; load ones code
121      MOVWF    lcd_reg
122      CALL     TYPE_CHAR     ; and output
123      MOVLW    ' '          ; load space code
124      MOVWF    lcd_reg
125      CALL     TYPE_CHAR     ; and output
126      MOVLW    'V'          ; load volts code
127      MOVWF    lcd_reg
128      CALL     TYPE_CHAR     ; and output
129      MOVLW    'o'          ; load volts code
130      MOVWF    lcd_reg
131      CALL     TYPE_CHAR     ; and output
132      MOVLW    'l'          ; load volts code
133      MOVWF    lcd_reg
134      CALL     TYPE_CHAR     ; and output
135      MOVLW    't'          ; load volts code
136      MOVWF    lcd_reg
137      CALL     TYPE_CHAR     ; and output
138      MOVLW    's'          ; load volts code
139      MOVWF    lcd_reg
140      CALL     TYPE_CHAR     ; and output
141      MOVLW    ' '          ; load space code
142      MOVWF    lcd_reg
143      CALL     TYPE_CHAR     ; and output
144      MOVLW    'F'          ; load F code
145      MOVWF    lcd_reg
146      CALL     TYPE_CHAR     ; and output
147      MOVLW    'r'          ; load r code
148      MOVWF    lcd_reg
149      CALL     TYPE_CHAR     ; and output
150      MOVLW    'o'          ; load o code
151      MOVWF    lcd_reg
152      CALL     TYPE_CHAR     ; and output

```

Figure 54: A listing of the code of function that displays the message and input for channel AD0

```

131      CALL    TYPE_CHAR    ; and output
132      MOVLW   'l'          ; load volts code
133      MOVWF   lcd_reg
134      CALL    TYPE_CHAR    ; and output
135      MOVLW   't'          ; load volts code
136      MOVWF   lcd_reg
137      CALL    TYPE_CHAR    ; and output
138      MOVLW   's'          ; load volts code
139      MOVWF   lcd_reg
140      CALL    TYPE_CHAR    ; and output
141      MOVLW   ' '          ; load space code
142      MOVWF   lcd_reg
143      CALL    TYPE_CHAR    ; and output
144      MOVLW   'F'          ; load F code
145      MOVWF   lcd_reg
146      CALL    TYPE_CHAR    ; and output
147      MOVLW   'r'          ; load r code
148      MOVWF   lcd_reg
149      CALL    TYPE_CHAR    ; and output
150      MOVLW   'o'          ; load o code
151      MOVWF   lcd_reg
152      CALL    TYPE_CHAR    ; and output
153      MOVLW   'm'          ; load m code
154      MOVWF   lcd_reg
155      CALL    TYPE_CHAR    ; and output
156      MOVLW   ' '          ; load space code
157      MOVWF   lcd_reg
158      CALL    TYPE_CHAR    ; and output
159
160      CALL    SET_ADDR     ; to skip a line
161
162      MOVLW   'T'          ; load T code
163      MOVWF   lcd_reg
164      CALL    TYPE_CHAR    ; and output
165      MOVLW   'l'          ; load l code
166      MOVWF   lcd_reg
167      CALL    TYPE_CHAR    ; and output
168      BTFSC   PORTB,RB0
169      CALL    CLEAR
170      NOP
171      RETURN                ; done

```

Figure 55: A listing of the code of function that displays the message and input for channel AD0

```

173 LCD2
174 BTFSS PORTB,RB0
175 CALL SET_AN1
176 ; Output to display.....
177 MOVLW 080 ; code to home cursor
178 MOVWF cmd_reg
179 CALL TYPE_COM ; output it to display
180 ; Convert digits to ASCII and display.....
181 MOVLW 030 ; load ASCII offset
182 ADDWF huns ; convert hundreds to ASCII
183 ADDWF tens ; convert tens to ASCII
184 ADDWF ones ; convert ones to ASCII
185 MOVF huns,W ; load hundreds code
186 MOVWF lcd_reg
187 CALL TYPE_CHAR ; and send to display
188 MOVLW '.' ; load point code
189 MOVWF lcd_reg
190 CALL TYPE_CHAR ; and output
191 MOVF tens,W ; load tens code
192 MOVWF lcd_reg
193 CALL TYPE_CHAR ; and output
194 MOVF ones,W ; load ones code
195 MOVWF lcd_reg
196 CALL TYPE_CHAR ; and output
197 MOVLW ' ' ; load space code
198 MOVWF lcd_reg
199 CALL TYPE_CHAR ; and output
200 MOVLW 'V' ; load volts code
201 MOVWF lcd_reg
202 CALL TYPE_CHAR ; and output
203 MOVLW '0' ; load volts code
204 MOVWF lcd_reg
205 CALL TYPE_CHAR ; and output
206 MOVLW '1' ; load volts code
207 MOVWF lcd_reg
208 CALL TYPE_CHAR ; and output
209 MOVLW 't' ; load volts code
210 MOVWF lcd_reg
211 CALL TYPE_CHAR ; and output
212 MOVLW 's' ; load volts code
213 MOVWF lcd_reg
214 CALL TYPE_CHAR ; and output
215 MOVLW ' ' ; load space code
216 MOVWF lcd_reg
217 CALL TYPE_CHAR ; and output
218 MOVLW 'F' ; load F code
219 MOVWF lcd_reg
220 CALL TYPE_CHAR ; and output
221 MOVLW 'r' ; load r code
222 MOVWF lcd_reg
223 CALL TYPE_CHAR ; and output
224 MOVLW 'o' ; load o code
225 MOVWF lcd_reg
226 CALL TYPE_CHAR ; and output

```

Figure 56: A listing of the code of function that displays the message and input for channel AD1

```

227      MOVLW    'm'           ; load m code
228      MOVWF    lcd_reg
229      CALL     TYPE_CHAR     ; and output
230      MOVLW    ' '           ; load space code
231      MOVWF    lcd_reg
232      CALL     TYPE_CHAR     ; and output
233
234      CALL     SET_ADDR      ; to skip a line
235
236      MOVLW    'T'           ; load T code
237      MOVWF    lcd_reg
238      CALL     TYPE_CHAR     ; and output
239      MOVLW    '2'           ; load 2 code
240      MOVWF    lcd_reg
241      CALL     TYPE_CHAR     ; and output
242      BTFSS    PORTB, RB0
243      CALL     CLEAR
244      RETURN                      ; done
245
246  inid      ; initialise the LCD
247      CALL     WAIT_15ms
248      CALL     INITIAL_LCD
249      RETURN
250
251  SET_AN0
252      MOVLW    B'01000001'
253      MOVWF    ADCON0
254      RETURN
255
256  SET_AN1
257      MOVLW    B'01001001'
258      MOVWF    ADCON0
259      RETURN
260
261      END

```

Figure 57: A listing of the code of function that displays the message and input for channel AD1, the function that starts the setup of the LCD Display and the functions that change the channel